CLD-105 · Orchestration de conteneurs avancé (Kubernetes/Git)

# Compte rendu – TP2

## Cluster kubeadm et clinique des pods

| Élément | Valeur |
|---|---|
| Étudiant | Qassim Messili |
| Formation | Mastère (Bac+5) – IPI Lyon |
| Date | 26 octobre 2026 |
| Nœuds | `k8s-cp` (10.2.100.225) et `k8s-w1` (10.2.100.219) – Ubuntu 26.04.1 LTS sur VMware ESXi |
| Versions | Kubernetes v1.34.12 · containerd 2.2.2 · Calico v3.30.2 · etcd 3.6.0 |

# Sommaire

*Si le sommaire est vide : clic droit → « Mettre à jour les champs » (ou F9 dans Word).*

# Introduction

Au TP1, K3s m'avait livré un cluster prêt à l'emploi en une seule commande. Ce deuxième TP prend le chemin inverse : monter un vrai cluster Kubernetes **à la main** avec kubeadm, l'outil d'amorçage officiel. Cela veut dire préparer moi-même le système (swap, modules noyau, sysctl), installer le runtime de conteneurs, choisir et déployer le réseau des pods, puis assembler un control plane et un worker.

Ce cluster n'est pas jetable : c'est lui qui accueillera l'application fil rouge dans les prochains TP, puis le GitOps, la sécurité et la supervision. J'ai donc pris le temps de comprendre chaque réglage plutôt que de simplement copier des commandes. La seconde moitié du TP est un exercice de diagnostic : trois pods volontairement cassés, à réparer avec une démarche méthodique.

## Architecture mise en place

| Rôle | VM ESXi | Hostname | IP | Ressources |
|---|---|---|---|---|
| Control plane | Kube qassim 1 | `k8s-cp` | 10.2.100.225/16 | 2 vCPU · 4 Go · 40 Go |
| Worker | Kube qassim 2 | `k8s-w1` | 10.2.100.219/16 | 2 vCPU · 4 Go · 40 Go |

| Réseau | Plage | Usage |
|---|---|---|
| Réseau des nœuds | 10.2.0.0/16 (passerelle et DNS 10.2.255.253) | Réseau de l'école |
| Pods (Calico) | 10.244.0.0/16, blocs /26 par nœud | Une IP par pod |
| Services | 10.96.0.0/12 | ClusterIP (10.96.0.1 = API) |

## Écarts avec l'énoncé

Mon environnement diffère un peu de celui prévu par le sujet. Je les liste ici une fois pour toutes :

| Énoncé | Réalisé | Raison |
|---|---|---|
| IP 192.168.56.10 / .11 | 10.2.100.225 / .219 | Plan d'adressage de l'ESXi de l'école |
| Réseau de détection Calico 192.168.56.0/24 | 10.2.0.0/16 | Réseau réel des nœuds |
| Utilisateur `ubuntu` | `qassim` | Compte créé à l'installation |
| Interface NAT + interne (VirtualBox) | Une seule interface `ens192` | VM ESXi sur « VM Network » |
| `sh -c` dans le pod etcd | Appel direct à `etcdctl` | L'image etcd 3.6 ne contient pas de shell |
| Fusion des kubeconfig K3s + kubeadm | Non réalisée | VM K3s du TP1 supprimée (voir partie 4) |

# Partie 0 – Création des machines virtuelles

J'ai commencé par supprimer l'ancienne VM du TP1 pour libérer des ressources, puis j'ai créé deux VM identiques sur l'ESXi. Le premier réflexe a été de corriger le nombre de CPU : l'assistant proposait **1 vCPU**, or `kubeadm init` refuse purement et simplement de démarrer en dessous de 2 (contrôle de pré-vol `NumCPU`). J'en ai profité pour passer le contrôleur disque en **VMware Paravirtual**, plus adapté à un Linux récent.

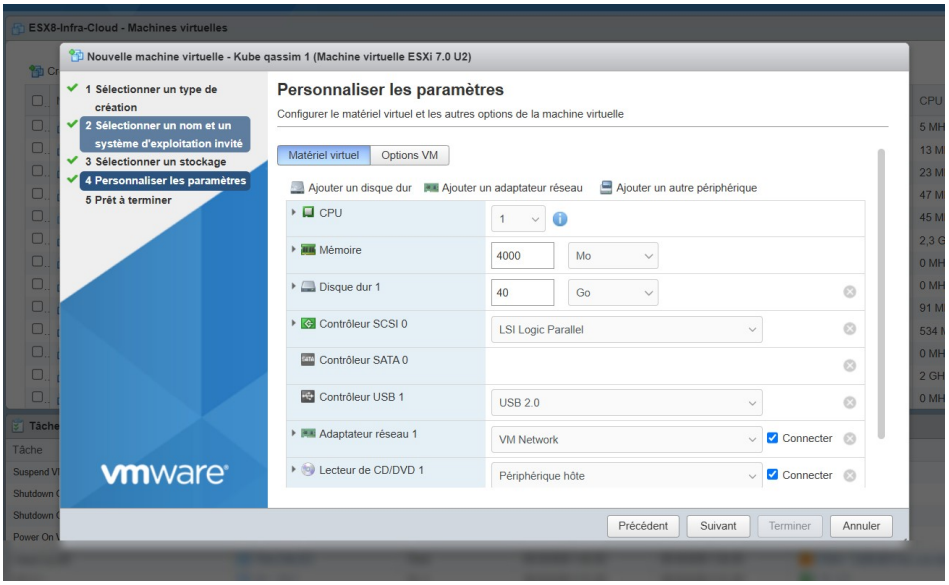



*Figure 1 – Paramètres proposés par défaut (1 CPU, à corriger)*




*Figure 2 – Récapitulatif final de la VM 1 : Paravirtual, VMXNET 3, ISO Ubuntu 26.04.1*

J'ai lancé les deux installations en parallèle. Je n'ai pas cloné la première VM : sans vCenter il n'y a pas de bouton « cloner », et un clone aurait gardé le même `machine-id`, ce qui peut provoquer des conflits DHCP et des identifiants de nœud en double côté Kubernetes. Lors de l'installation, je n'ai coché aucun snap (contrairement au TP1 où MicroK8s m'avait posé problème) et j'ai activé OpenSSH.

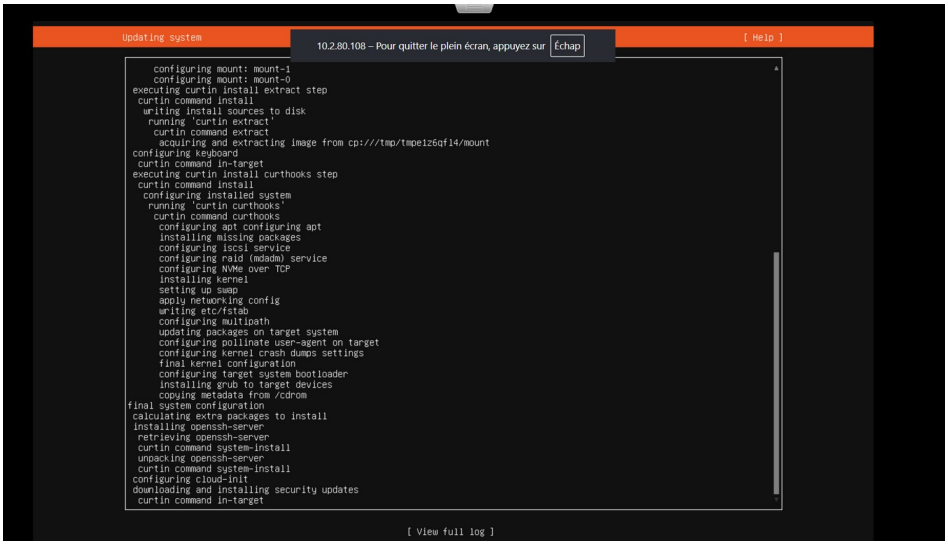



*Figure 3 – Fin de l'installation d'Ubuntu Server (installation d'OpenSSH, mises à jour)*

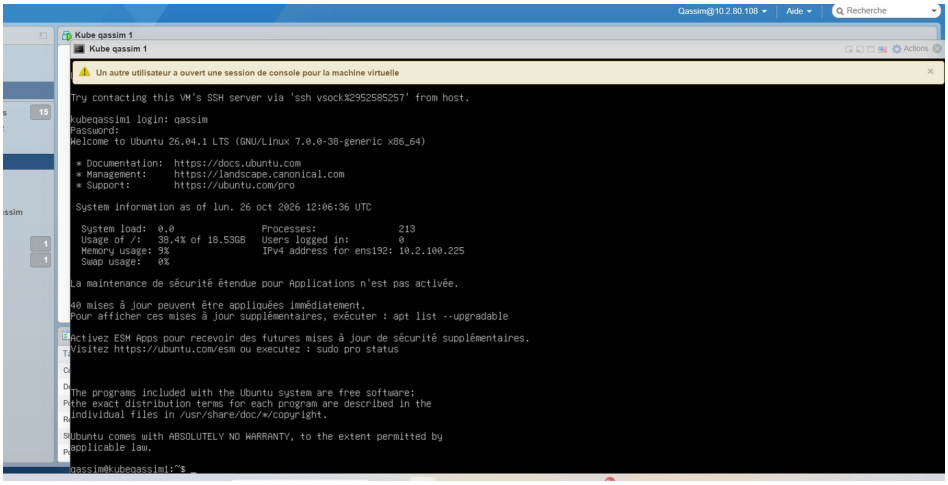



*Figure 4 – Premier démarrage : IP DHCP affichée sur ens192 (VM 1 – k8s-cp)*

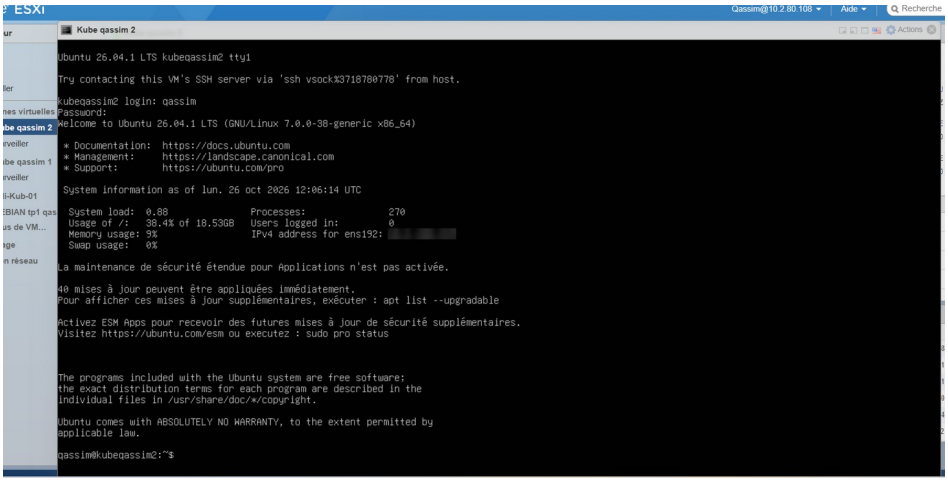



*Figure 5 – Premier démarrage : IP DHCP affichée sur ens192 (VM 2 – k8s-w1)*

## Préparation complémentaire : disque et IP fixe

Deux détails m'ont sauté aux yeux au premier démarrage. D'abord, `/` ne faisait que 18,5 Go alors que le disque en fait 40 : l'installateur Ubuntu n'alloue par défaut que la moitié du groupe LVM. Ensuite, les IP

étaient attribuées en **DHCP**. Or kubeadm grave l'adresse du control plane dans ses certificats : si elle change après un redémarrage, le cluster tombe. J'ai donc étendu le volume logique et figé les IP en statique avec netplan, sur les deux VM.

**VM 1 et VM 2**

```
sudo lvextend -r -l +100%FREE /dev/ubuntu-vg/ubuntu-lv

IF=ens192
ADDR=$(ip -4 -o addr show $IF | awk '{print $4}')
GW=$(ip route | awk '/default/{print $3; exit}')
DNS=$(resolvectl dns $IF | awk -F': ' '{print $2}' | tr ' ' ',')
echo "ADDR=$ADDR  GW=$GW  DNS=$DNS"
```

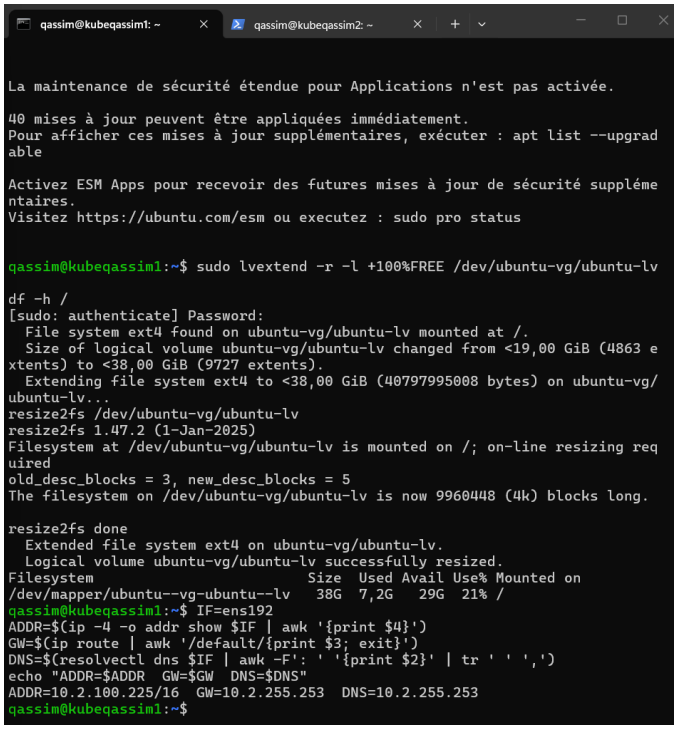



*Figure 6 – Volume étendu à 38 Go et relevé de la configuration réseau (VM 1 – k8s-cp)*

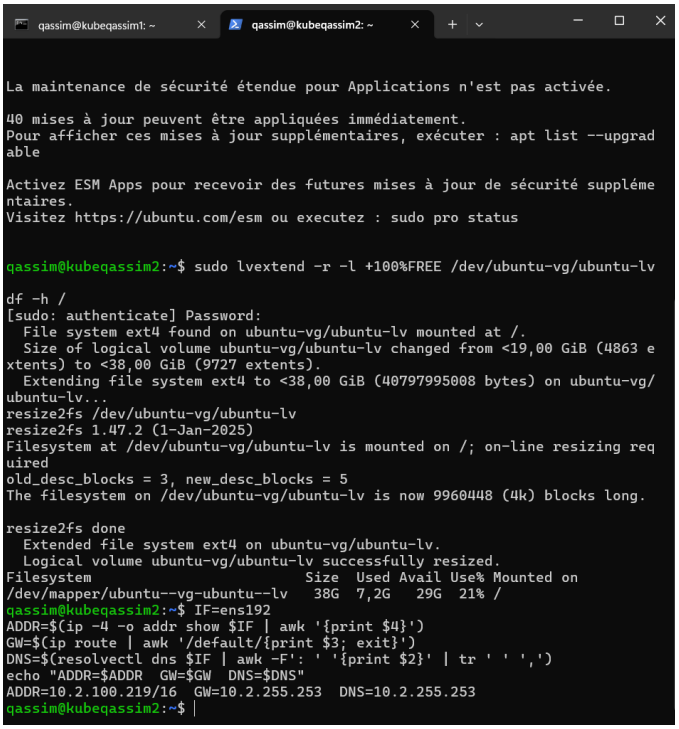



*Figure 7 – Volume étendu à 38 Go et relevé de la configuration réseau (VM 2 – k8s-w1)*

**VM 1 et VM 2**

```
sudo tee /etc/netplan/99-static.yaml >/dev/null <<EOF
network:
  version: 2
  ethernets:
    $IF:
      dhcp4: false
      addresses: [$ADDR]
      routes:
        - to: default
          via: $GW
      nameservers:
        addresses: [$DNS]
EOF
echo 'network: {config: disabled}' | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg
sudo rm -f /etc/netplan/50-cloud-init.yaml
sudo chmod 600 /etc/netplan/99-static.yaml
sudo netplan apply
```

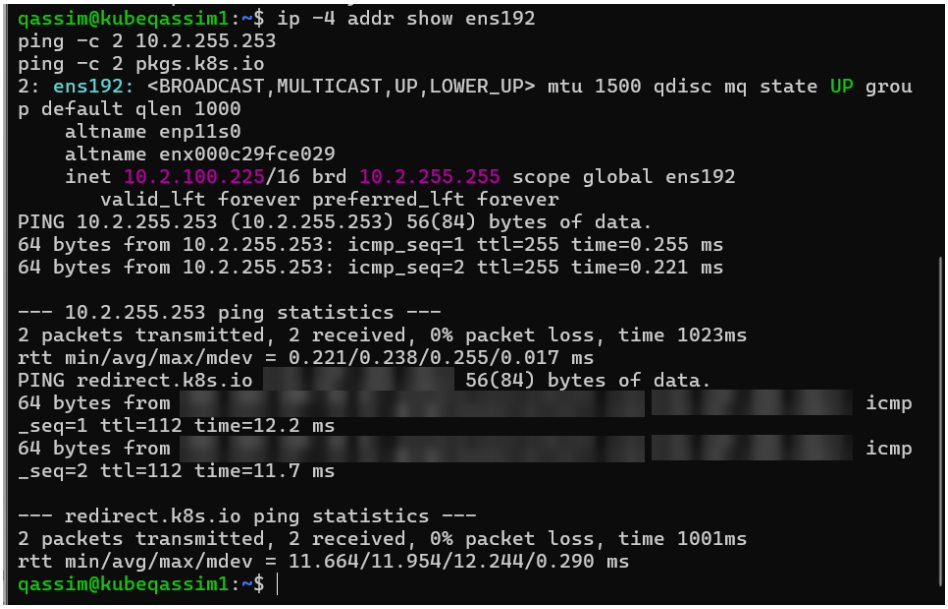



*Figure 8 – IP fixe (valid_lft forever) et accès Internet vérifié (VM 1 – k8s-cp)*

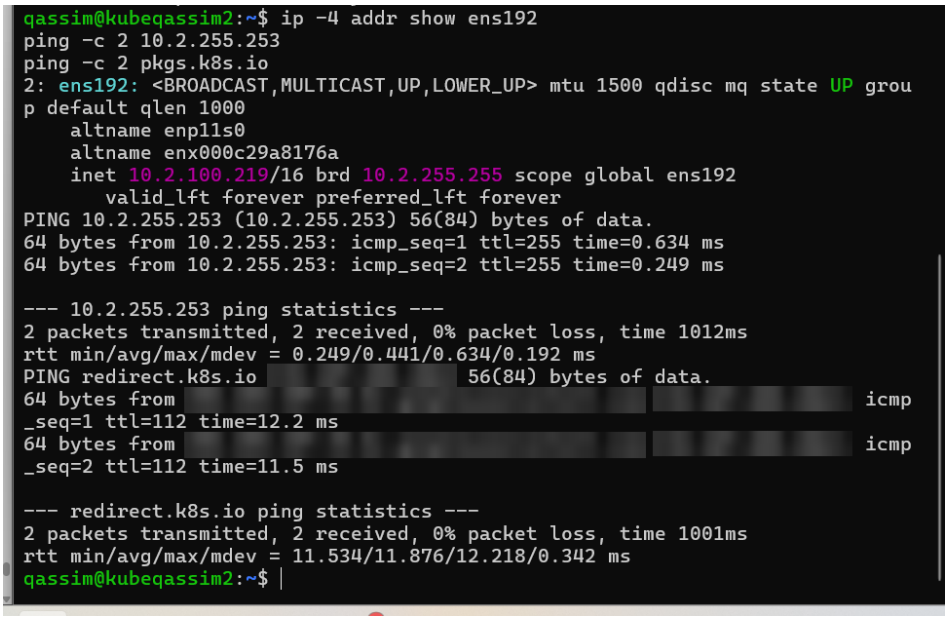



*Figure 9 – IP fixe (valid_lft forever) et accès Internet vérifié (VM 2 – k8s-w1)*

Le mot `dynamic` a disparu de la sortie de `ip addr` et la durée de validité est passée à `forever` : l'adresse est bien statique. La passerelle et `pkgs.k8s.io` répondent, ce qui sera nécessaire pour télécharger les paquets.

# Partie 1 – Préparer les deux nœuds

Toutes les étapes de cette partie sont jouées à l'identique sur les deux VM, à l'exception du nom d'hôte et de l'IP du kubelet. J'ai travaillé avec deux onglets SSH côte à côte dans Windows Terminal.

## 1. Nom d'hôte et résolution des noms

```
CP_IP=10.2.100.225
W1_IP=10.2.100.219
sudo hostnamectl set-hostname k8s-cp     # sur la VM 2 : k8s-w1
printf "$CP_IP k8s-cp\n$W1_IP k8s-w1\n" | sudo tee -a /etc/hosts
hostname
cat /etc/hosts
ping -c 2 k8s-cp
ping -c 2 k8s-w1
```

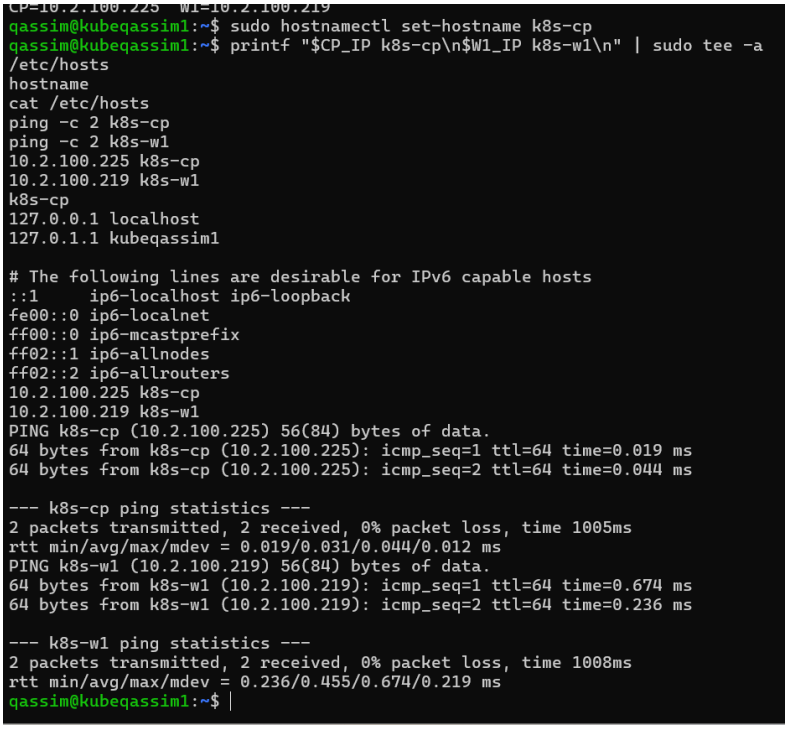



*Figure 10 – Hostnames définis et résolution croisée par /etc/hosts (VM 1 – k8s-cp)*

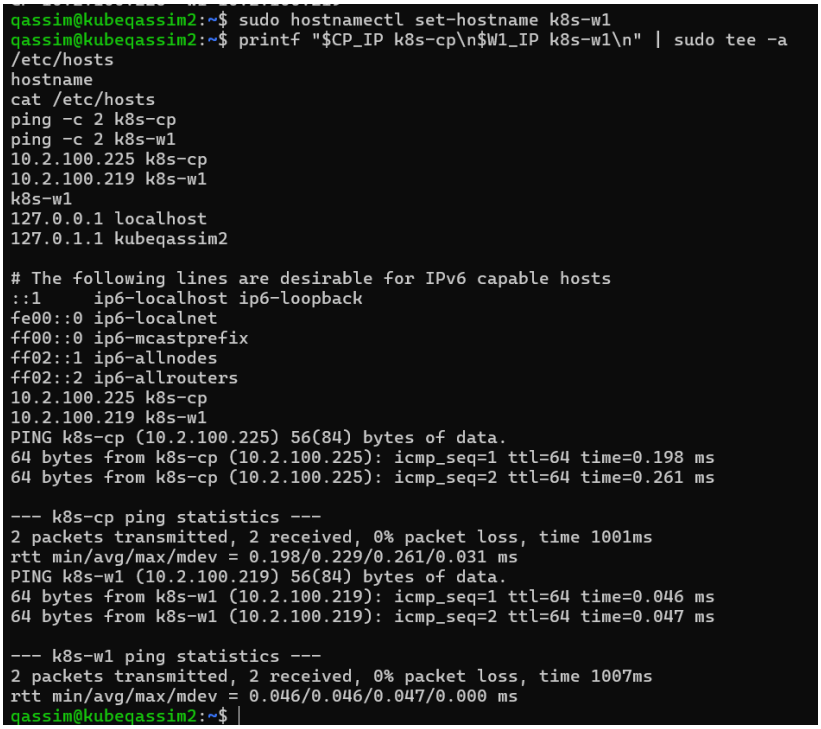



*Figure 11 – Hostnames définis et résolution croisée par /etc/hosts (VM 2 – k8s-w1)*

Les deux nœuds se joignent par leur nom. J'ai ensuite corrigé la ligne `127.0.1.1` qui portait encore l'ancien nom de la machine (`kubeqassim1`), pour éviter l'avertissement « unable to resolve host » de sudo.

## 2. Désactivation du swap

```
sudo swapoff -a
sudo sed -i '/\sswap\s/s/^/#/' /etc/fstab
free -h
grep swap /etc/fstab
```

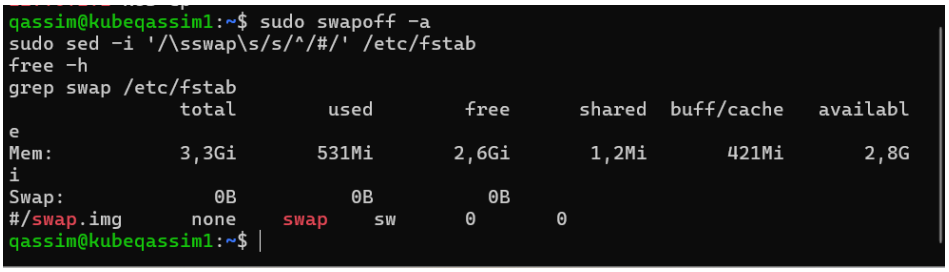



*Figure 12 – Swap à 0B et ligne /swap.img commentée (VM 1 – k8s-cp)*




*Figure 13 – Swap à 0B et ligne /swap.img commentée (VM 2 – k8s-w1)*

**Pourquoi désactiver le swap ?** Le kubelet refuse par défaut de démarrer si du swap est actif. Kubernetes a besoin d'une consommation mémoire prévisible pour décider où placer les pods et pour faire respecter leurs limites. Avec du swap, un conteneur peut dépasser sa limite sans que personne ne s'en rende compte, et les performances deviennent imprévisibles. La commande `swapoff` agit tout de suite, la modification de `/etc/fstab` garantit que cela tient après un redémarrage.

## 3. Modules noyau et paramètres sysctl

```
printf 'overlay\nbr_netfilter\n' | sudo tee /etc/modules-load.d/k8s.conf
sudo modprobe overlay && sudo modprobe br_netfilter
cat <<'EOF' | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

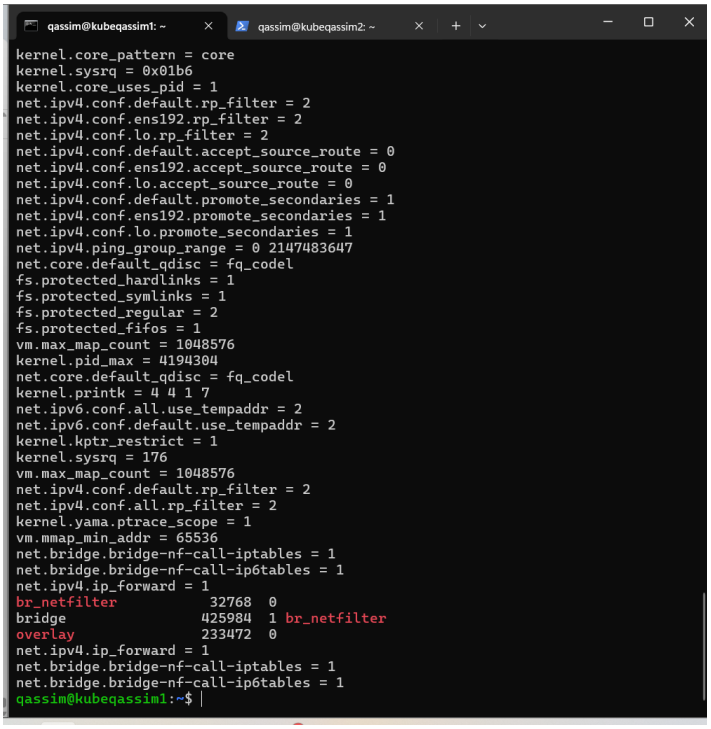



*Figure 14 – Modules chargés et trois paramètres à 1 (VM 1 – k8s-cp)*

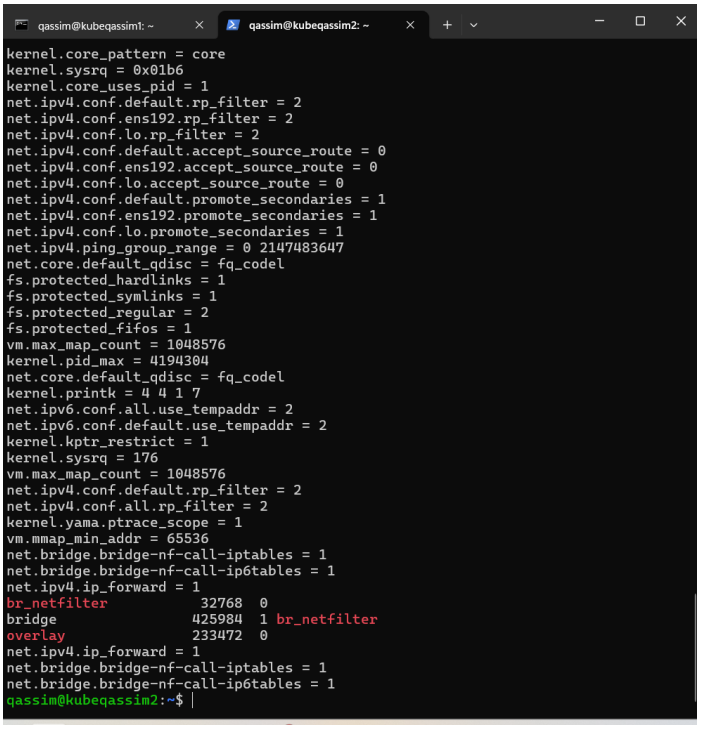



*Figure 15 – Modules chargés et trois paramètres à 1 (VM 2 – k8s-w1)*

| Réglage | À quoi il sert |
|---|---|
| `overlay` | Système de fichiers en couches utilisé par containerd pour empiler les couches des images. |
| `br_netfilter` | Fait passer le trafic des bridges Linux (réseau des pods) par iptables. |
| `bridge-nf-call-iptables / ip6tables` | Applique les règles iptables (Services, NetworkPolicies) au trafic qui traverse les bridges. |
| `ip_forward` | Autorise le nœud à router des paquets d'une interface à l'autre (pod ↔ pod, pod ↔ extérieur). |

Les fichiers sont placés dans `/etc/modules-load.d/` et `/etc/sysctl.d/`, les réglages persistent donc au redémarrage.

## 4. Installation de containerd

```
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
grep SystemdCgroup /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
```

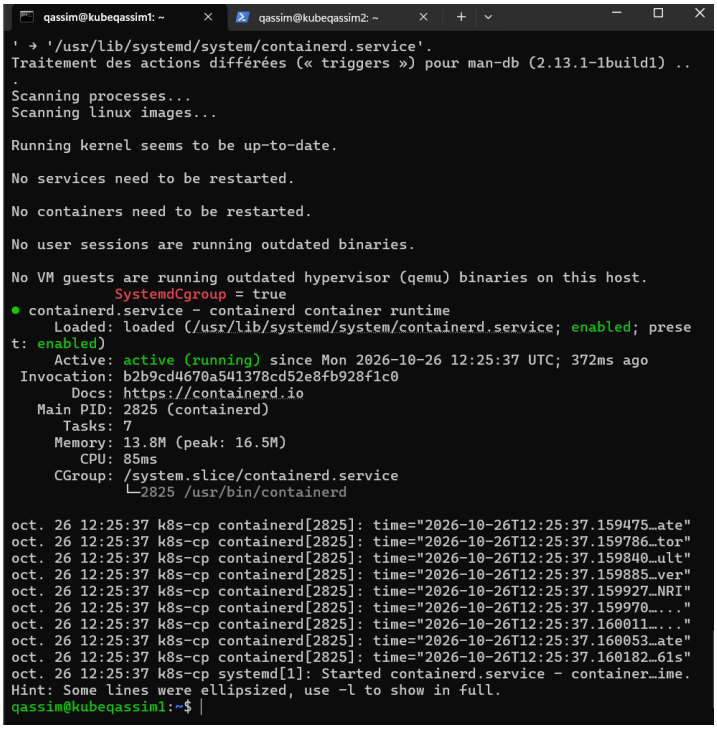



*Figure 16 – containerd actif avec SystemdCgroup = true (VM 1 – k8s-cp)*

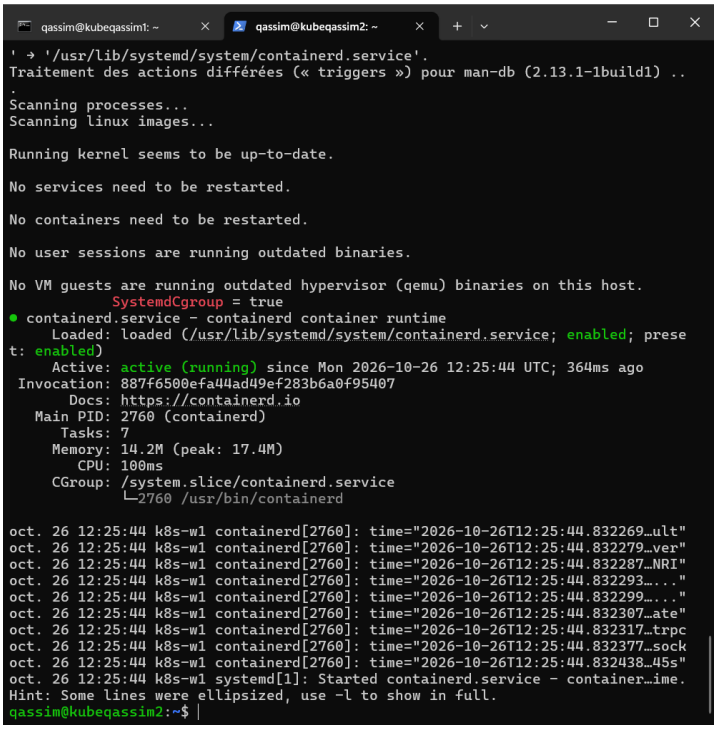



*Figure 17 – containerd actif avec SystemdCgroup = true (VM 2 – k8s-w1)*

> **Pourquoi `SystemdCgroup = true` est-il indispensable ?** Ubuntu confie la gestion des cgroups (le mécanisme du noyau qui limite et comptabilise CPU et mémoire) à systemd. kubeadm configure le kubelet avec `cgroupDriver: systemd`. Si containerd utilisait un autre pilote (cgroupfs), deux gestionnaires se disputeraient la même hiérarchie : les limites deviendraient incohérentes et des pods système redémarreraient au hasard, surtout sous charge. Le kubelet et le runtime doivent parler le même langage.

## 5. Paquets Kubernetes

```
sudo apt install -y apt-transport-https ca-certificates curl gpg
K8S_VERSION=v1.34
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/$K8S_VERSION/deb/Release.key \
  | sudo gpg --dearmor --yes -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/$K8S_VERSION/deb/ /" \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update && sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

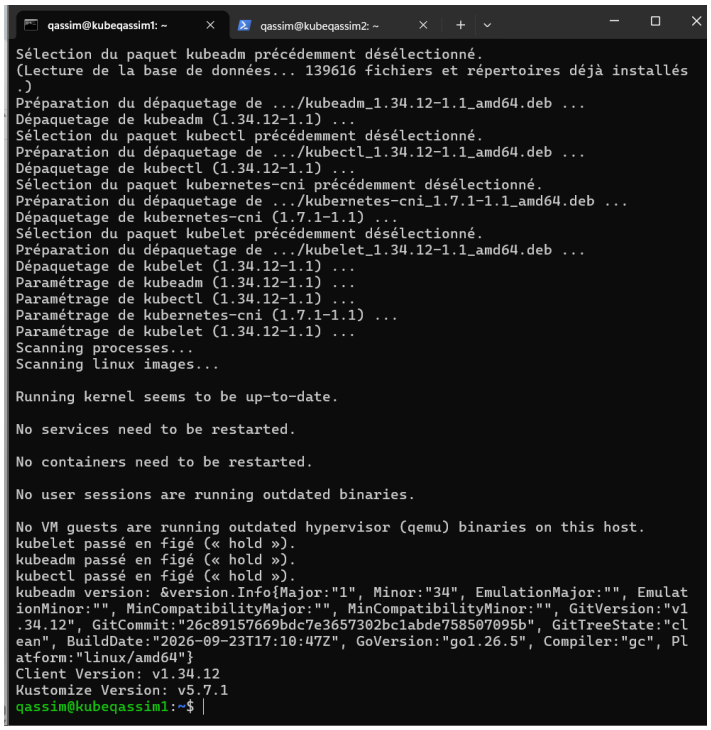



*Figure 18 – kubelet, kubeadm et kubectl v1.34.12 figés (VM 1 – k8s-cp)*

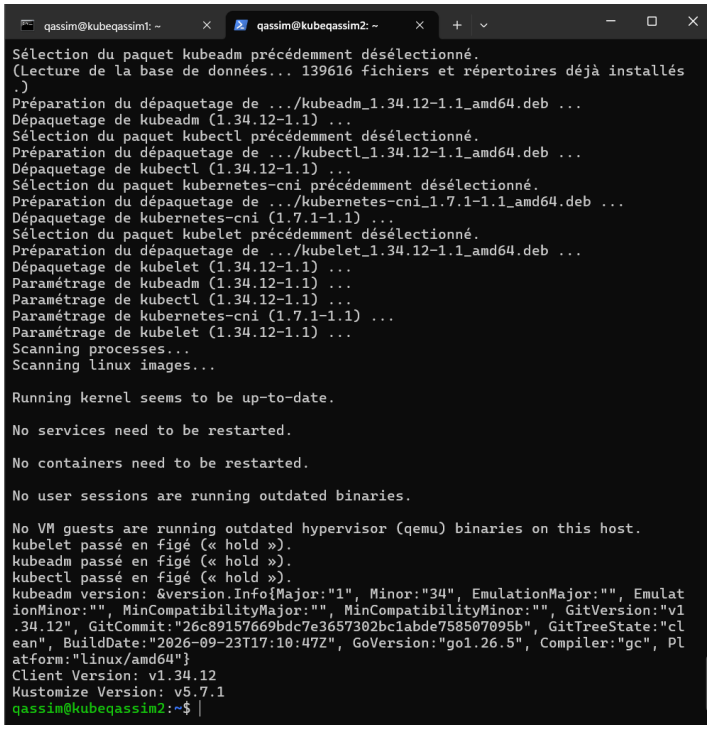



*Figure 19 – kubelet, kubeadm et kubectl v1.34.12 figés (VM 2 – k8s-w1)*

Les trois outils sont installés en **v1.34.12** et passés en `hold`. Sans ce gel, un simple `apt upgrade` pourrait mettre à jour Kubernetes dans mon dos et créer un écart de versions entre les nœuds. Les montées de version doivent rester volontaires et se faire nœud par nœud avec `kubeadm upgrade`.

## 6. Le kubelet qui redémarre en boucle

Juste après l'installation, le kubelet était `inactive (dead)` : le paquet active le service au démarrage mais ne le lance pas. Je l'ai donc démarré à la main pour observer le comportement décrit dans l'énoncé.

```
sudo systemctl start kubelet
sleep 5
systemctl status kubelet --no-pager
sudo journalctl -u kubelet --no-pager -n 5
```

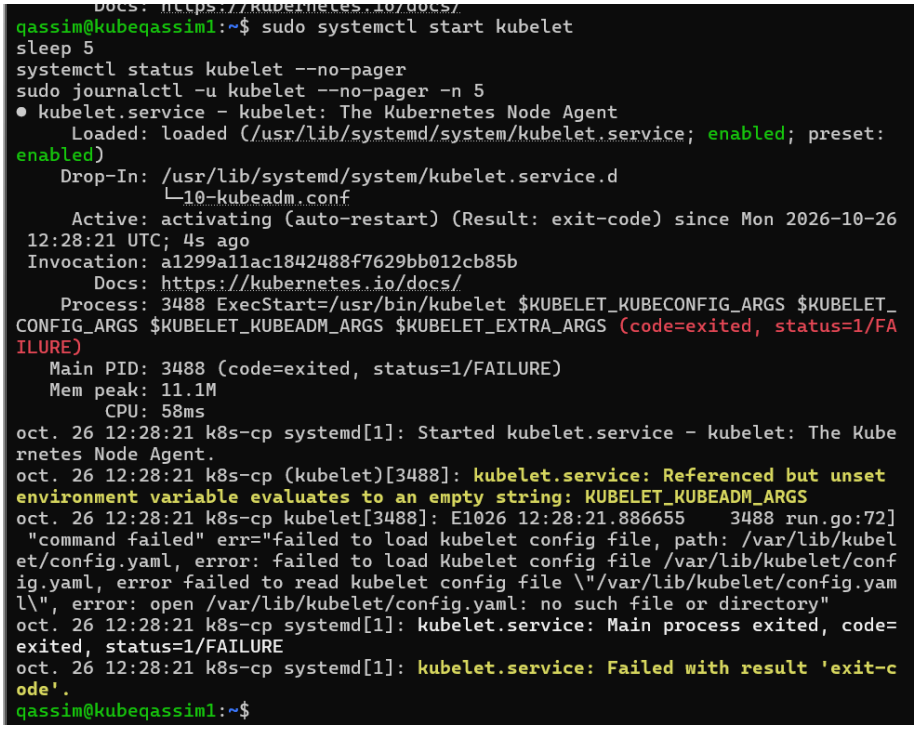



*Figure 20 – kubelet en activating (auto-restart), status=1/FAILURE (VM 1 – k8s-cp)*

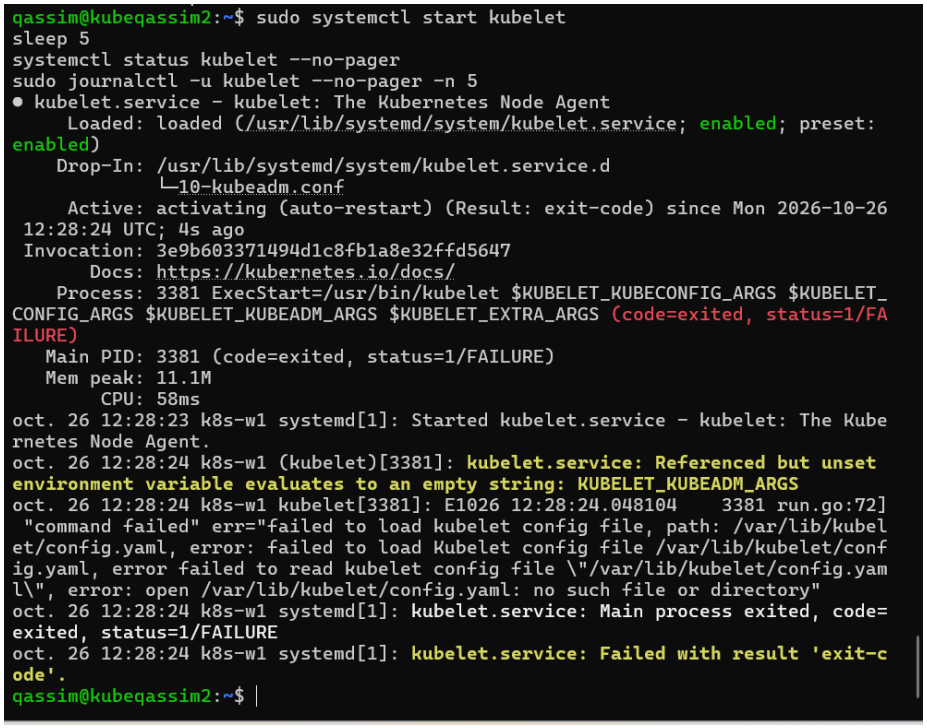



*Figure 21 – kubelet en activating (auto-restart), status=1/FAILURE (VM 2 – k8s-w1)*

> **Pourquoi cette boucle est-elle normale ?** Les logs le disent clairement : `failed to load kubelet config file /var/lib/kubelet/config.yaml: no such file or directory`. Le kubelet a besoin de ce fichier, de ses certificats et des options de `kubeadm-flags.env` (d'où l'avertissement sur `KUBELET_KUBEADM_ARGS` vide). Tout cela n'existe pas encore : c'est `kubeadm init` ou `kubeadm join` qui va l'écrire. En attendant, le kubelet s'arrête en erreur et systemd le relance toutes les 10 secondes.

## 7. Adresse annoncée par le kubelet

```
echo "KUBELET_EXTRA_ARGS=--node-ip=10.2.100.225" | sudo tee /etc/default/kubelet
# VM 1
echo "KUBELET_EXTRA_ARGS=--node-ip=10.2.100.219" | sudo tee /etc/default/kubelet
# VM 2
```

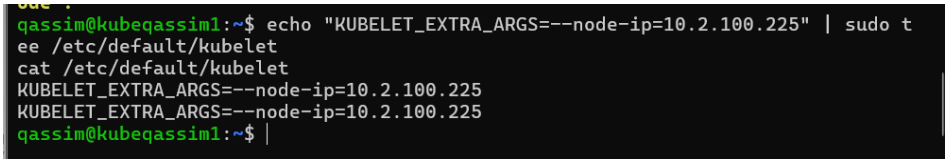



*Figure 22 – IP du nœud fixée pour le kubelet (VM 1 – k8s-cp)*

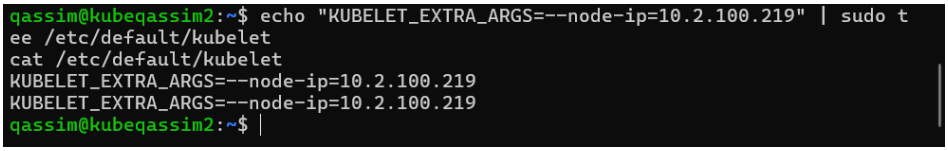



*Figure 23 – IP du nœud fixée pour le kubelet (VM 2 – k8s-w1)*

Sur une VM à deux interfaces, le kubelet choisirait celle de la route par défaut (souvent la NAT) et annoncerait une adresse injoignable par les autres nœuds : `kubectl logs` et `exec` échoueraient. Mes VM n'ont qu'une interface, mais fixer l'adresse rend la configuration déterministe, même si une carte est ajoutée plus tard.

# Partie 2 – Initialiser le control plane

Cette partie se déroule uniquement sur `k8s-cp`. J'ai redirigé la sortie de `kubeadm init` dans un fichier journal pour pouvoir la relire tranquillement.

**VM 1**

```
sudo kubeadm init \
  --apiserver-advertise-address=10.2.100.225 \
  --pod-network-cidr=10.244.0.0/16 \
  --node-name=k8s-cp | tee ~/kubeadm-init.log
```

```
qassim@kubeqassim1:~$ sudo kubeadm init \
  --apiserver-advertise-address=10.2.100.225 \
  --pod-network-cidr=10.244.0.0/16 \
  --node-name=k8s-cp | tee ~/kubeadm-init.log
I1026 12:30:45.620061    3672 version.go:260] remote version is much newer: v1.
37.1; falling back to: stable-1.34
[init] Using Kubernetes version: v1.34.12
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your int
ernet connection
[preflight] You can also perform this action beforehand using 'kubeadm config i
mages pull'
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [k8s-cp kubernetes kuber
netes.default kubernetes.default.svc kubernetes.default.svc.cluster.local] and
IPs [10.96.0.1 10.2.100.225]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [k8s-cp localhost] and
 IPs [10.2.100.225 127.0.0.1 ::1]
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [k8s-cp localhost] and I
Ps [10.2.100.225 127.0.0.1 ::1]
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
[kubeconfig] Using kubeconfig folder "/etc/kubernetes"
[kubeconfig] Writing "admin.conf" kubeconfig file
[kubeconfig] Writing "super-admin.conf" kubeconfig file
[kubeconfig] Writing "kubelet.conf" kubeconfig file
[kubeconfig] Writing "controller-manager.conf" kubeconfig file
[kubeconfig] Writing "scheduler.conf" kubeconfig file
[etcd] Creating static Pod manifest for local etcd in "/etc/kubernetes/manifest
s"
```

*Figure 24 – Début de kubeadm init : pré-vol, certificats, kubeconfigs*

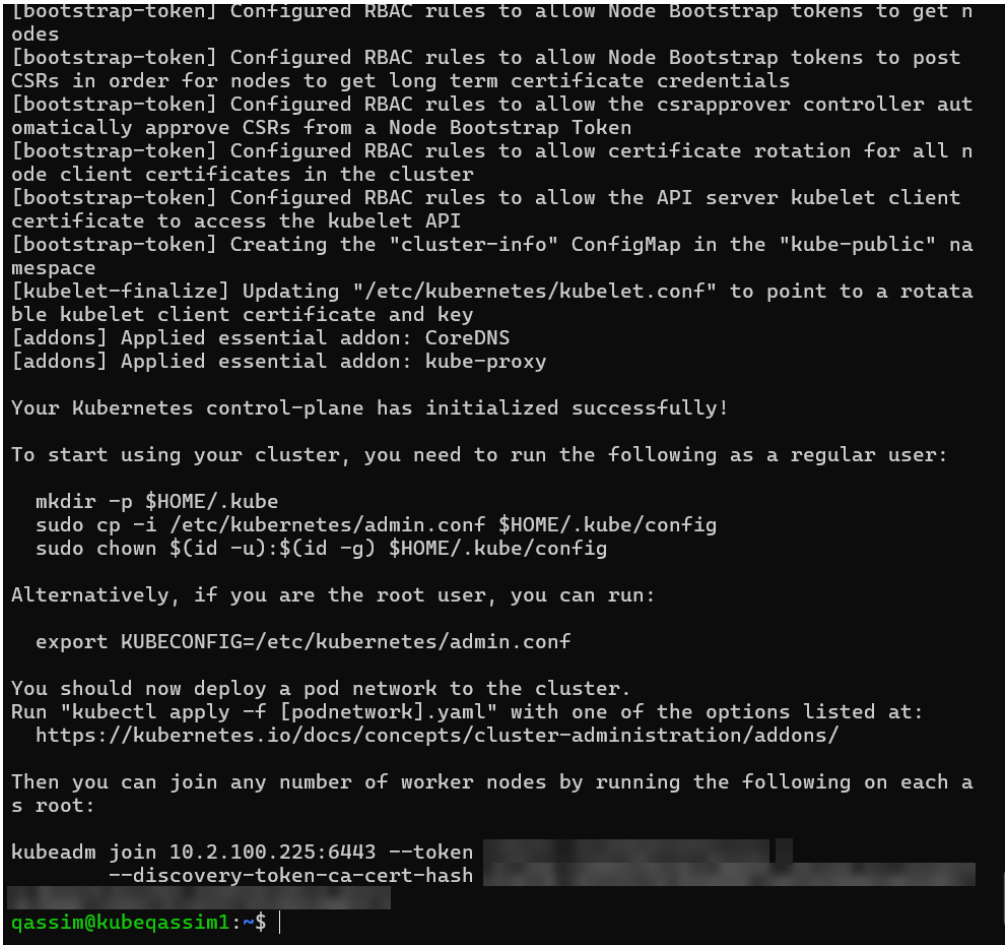



*Figure 25 – Fin de kubeadm init et commande kubeadm join générée*

En lisant la sortie, on suit l'enchaînement des phases :

| Phase | Ce qui se passe |
|---|---|
| `[preflight]` | Vérifications (CPU, swap, ports…) et téléchargement des images. kubeadm a vu qu'une v1.37 existait mais est resté sur `stable-1.34`, cohérent avec ses paquets. |
| `[certs]` | Création de toute la PKI dans `/etc/kubernetes/pki`. Le certificat de l'API est valable pour `k8s-cp`, `kubernetes.default…`, 10.96.0.1 et 10.2.100.225. |
| `[kubeconfig]` | Écriture de `admin.conf`, `super-admin.conf`, `kubelet.conf`, `controller-manager.conf` et `scheduler.conf` dans `/etc/kubernetes`. |
| `[etcd]` / `[control-plane]` | Manifests des pods statiques déposés dans `/etc/kubernetes/manifests`. |
| `[upload-config]` | Configuration stockée dans la ConfigMap `kubeadm-config` (que le worker relira au join). |
| `[bootstrap-token]` | Jeton de join et règles RBAC pour que les nouveaux nœuds obtiennent leur certificat. |
| `[addons]` | Déploiement de CoreDNS et kube-proxy. |

J'ai mis de côté la commande de join (jeton et empreinte `sha256` de la CA), valable 24 heures.

> **Pourquoi 10.244.0.0/16 plutôt que le défaut de Calico (192.168.0.0/16) ?** Le réseau des pods doit être totalement disjoint des réseaux physiques joignables. Si les pods recevaient des adresses qui recouvrent celles des nœuds, les routes du CNI captureraient le trafic entre machines et l'API deviendrait injoignable, avec des symptômes très déroutants. Ici les nœuds sont en 10.2.0.0/16 et les Services en 10.96.0.0/12 : 10.244.0.0/16 ne chevauche ni l'un ni l'autre, et évite au passage les 192.168.x.x très fréquents sur les box et les VPN.

## Configurer kubectl et observer le cluster

VM 1

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
kubectl get pods -A
kubectl describe node k8s-cp | grep -A6 Conditions
```

```
qassim@kubeqassim1:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
qassim@kubeqassim1:~$ kubectl get nodes
kubectl get pods -A
kubectl describe node k8s-cp | grep -A6 Conditions
NAME     STATUS     ROLES           AGE    VERSION
k8s-cp   NotReady   control-plane   103s   v1.34.12
NAMESPACE     NAME                                READY   STATUS    RESTARTS   AGE
kube-system   coredns-66bc5c9577-8c2gn            0/1     Pending   0          95s
kube-system   coredns-66bc5c9577-8ms7g            0/1     Pending   0          95s
kube-system   etcd-k8s-cp                         1/1     Running   0          104
s
kube-system   kube-apiserver-k8s-cp               1/1     Running   0          104
s
kube-system   kube-controller-manager-k8s-cp      1/1     Running   0          101
s
kube-system   kube-proxy-7xqzn                    1/1     Running   0          95s
kube-system   kube-scheduler-k8s-cp               1/1     Running   0          104
s
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime
            Reason                       Message
  ----             ------  -----------------                 ------------------
            ------                       -------
  MemoryPressure   False   Mon, 26 Oct 2026 12:30:56 +0000   Mon, 26 Oct 2026 1
2:30:50 +0000   KubeletHasSufficientMemory   kubelet has sufficient memory avai
lable
  DiskPressure     False   Mon, 26 Oct 2026 12:30:56 +0000   Mon, 26 Oct 2026 1
2:30:50 +0000   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Mon, 26 Oct 2026 12:30:56 +0000   Mon, 26 Oct 2026 1
2:30:50 +0000   KubeletHasSufficientPID      kubelet has sufficient PID availab
le
  Ready            False   Mon, 26 Oct 2026 12:30:56 +0000   Mon, 26 Oct 2026 1
2:30:50 +0000   KubeletNotReady              container runtime network not read
y: NetworkReady=false reason:NetworkPluginNotReady message:Network plugin retur
ns error: cni plugin not initialized
qassim@kubeqassim1:~$
```

*Figure 26 – Nœud NotReady et CoreDNS en Pending avant l'installation du CNI*

> **Quel statut et pourquoi ?** Le nœud est **NotReady** : la condition indique `NetworkReady=false … cni plugin not initialized`. Aucun plugin réseau n'est encore installé. Les deux pods **CoreDNS** restent **Pending** car ils ont besoin d'une IP de pod, que seul un CNI peut fournir. En revanche etcd, l'API server, le controller-manager, le scheduler et kube-proxy sont **Running** : ils tournent en `hostNetwork`, directement sur l'IP du nœud, et n'ont donc pas besoin du CNI.

## Explorer manifests et PKI

**VM 1**

```
ls -l /etc/kubernetes/manifests
sudo ls -l /etc/kubernetes/pki /etc/kubernetes/pki/etcd
kubectl get pods -n kube-system -o wide
```

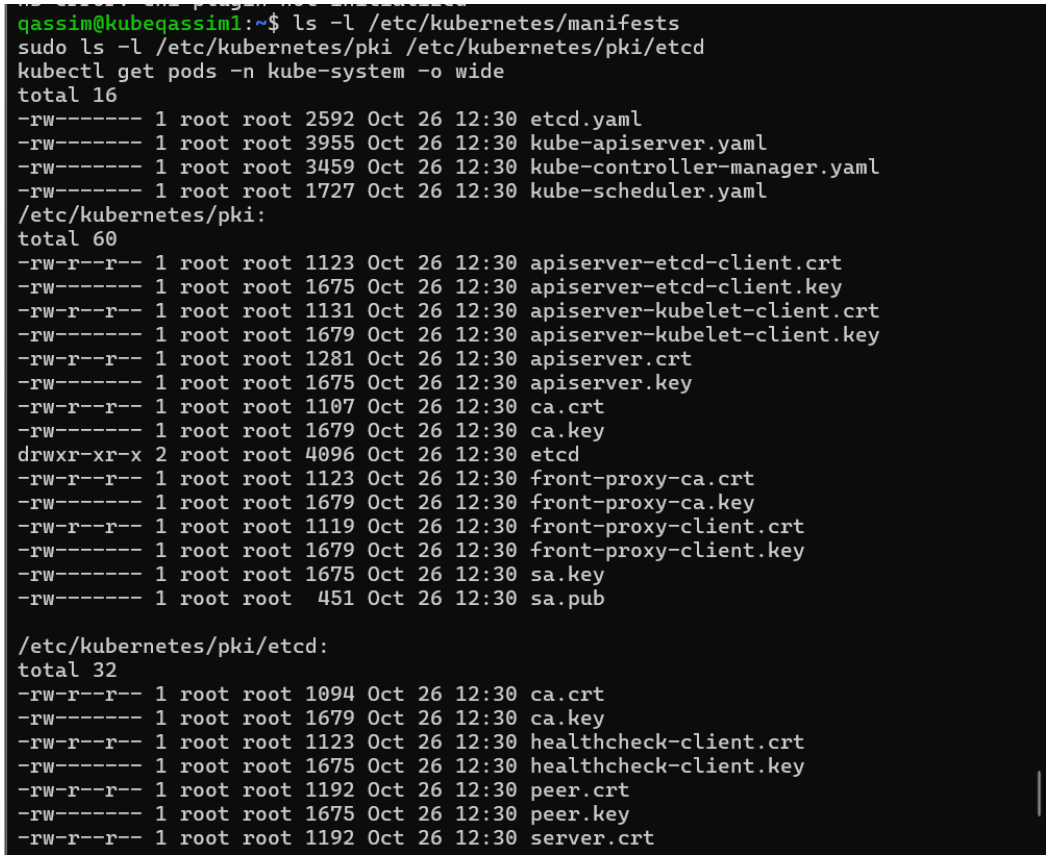



*Figure 27 – Les quatre manifests statiques et le contenu de la PKI*

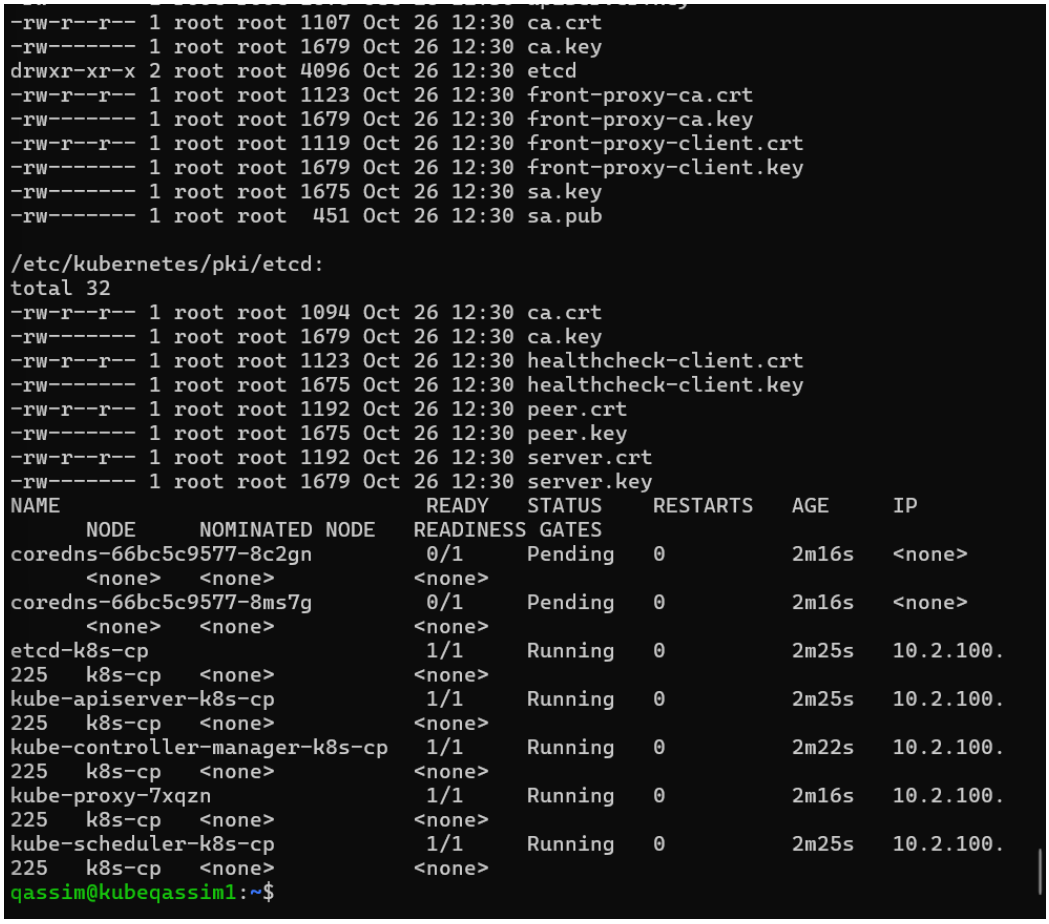



*Figure 28 – Pods du control plane portant l'IP du nœud et le suffixe -k8s-cp*

> **Comment sont lancés les composants du control plane ?** Ce sont des **pods statiques**. Le kubelet surveille `/etc/kubernetes/manifests` (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) et lance ces pods lui-même via containerd, sans passer par l'API server. C'est logique : l'API server fait partie de ces pods, il ne peut donc pas exister avant eux. Le kubelet publie ensuite des « pods miroirs » dans l'API, reconnaissables à leur suffixe `-k8s-cp` et à l'IP du nœud (10.2.100.225). On ne peut pas les supprimer avec kubectl : il faut retirer le fichier du dossier.

| Fichiers de la PKI | Rôle |
|---|---|
| `ca.crt / ca.key` | Autorité de certification qui signe tous les certificats du cluster |
| `apiserver.crt` | Certificat serveur de l'API |
| `apiserver-kubelet-client` | L'API s'authentifie auprès des kubelets (logs, exec) |
| `apiserver-etcd-client` | L'API s'authentifie auprès d'etcd |
| `front-proxy-*` | Agrégation d'API (metrics-server par exemple) |
| `sa.key / sa.pub` | Signature et vérification des jetons de ServiceAccount |
| `etcd/*` | PKI propre à etcd : serveur, réplication (peer), sondes de santé |

On remarque que les clés privées sont en permissions 600 (lisibles par root uniquement) tandis que les certificats publics sont en 644.

## Version déclarative : kubeadm-config.yaml

Pour finir cette partie, j'ai réécrit mon initialisation sous forme de fichier de configuration. Je ne l'ai pas rejoué, mais je l'ai fait valider par kubeadm.

**~/kubeadm-config.yaml**

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 10.2.100.225
  bindPort: 6443
nodeRegistration:
  name: k8s-cp
  criSocket: unix:///run/containerd/containerd.sock
  kubeletExtraArgs:
    - name: node-ip
      value: 10.2.100.225
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
kubernetesVersion: v1.34.12
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
  dnsDomain: cluster.local
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
```

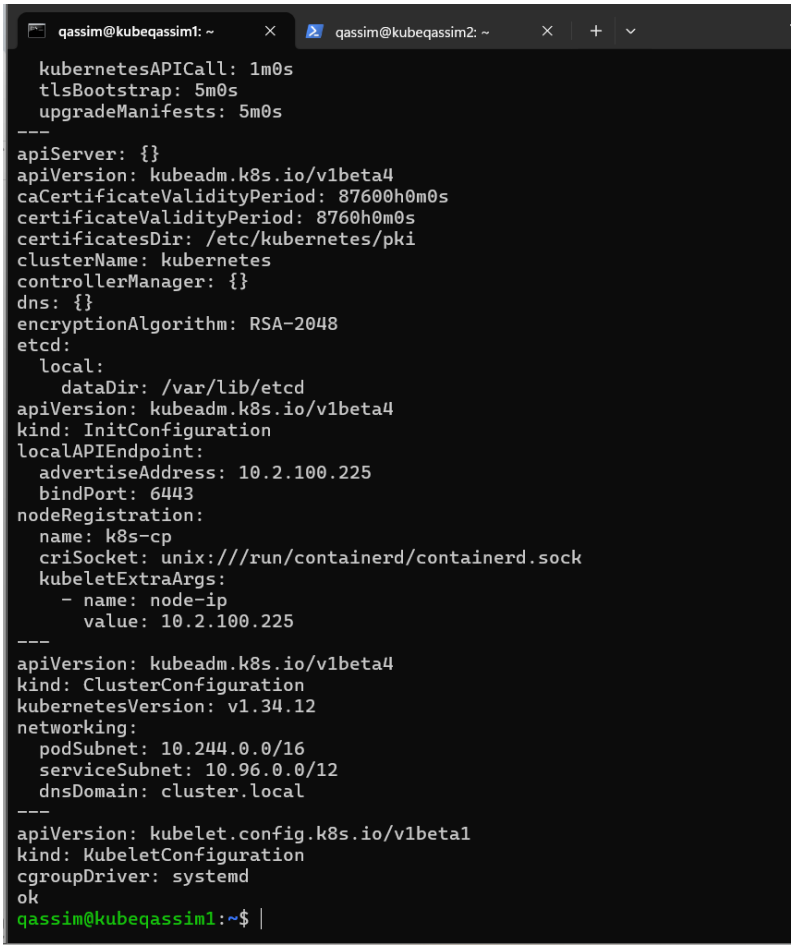



*Figure 29 – Fichier kubeadm-config.yaml validé (« ok »)*

**InitConfiguration** contient ce qui est propre au nœud (adresse annoncée, nom, socket du runtime), **ClusterConfiguration** ce qui concerne tout le cluster (version, réseaux) et **KubeletConfiguration** le pilote de cgroups, aligné sur containerd. L'intérêt est d'avoir une configuration versionnable dans Git et reproductible, au lieu d'une longue ligne d'options. La sortie de `print init-defaults` rappelle aussi des valeurs utiles : CA valable 10 ans, certificats valables 1 an.

## Partie 3 – Installer Calico

VM 1

```
CALICO_VERSION=v3.30.2
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/$CALICO_VERSION/
manifests/tigera-operator.yaml
```

> **Pourquoi `create` plutôt que `apply` ?** Le manifest de l'opérateur Tigera contient des CRD énormes. En mode client, `kubectl apply` recopie l'objet entier dans l'annotation `last-applied-configuration`, qui est limitée à 256 Ko : on obtient l'erreur `metadata.annotations: Too long`. `create` n'écrit pas cette annotation. L'autre solution serait `kubectl apply --server-side`.

J'ai ensuite décrit l'installation souhaitée dans une ressource `Installation`, en adaptant la détection d'adresse à mon réseau :

~/calico-custom-resources.yaml

```
apiVersion: operator.tigera.io/v1
kind: Installation
metadata:
  name: default
spec:
  calicoNetwork:
    ipPools:
      - name: default-ipv4-ippool
        blockSize: 26
        cidr: 10.244.0.0/16
        encapsulation: VXLANCrossSubnet
        natOutgoing: Enabled
        nodeSelector: all()
    nodeAddressAutodetectionV4:
      cidrs:
        - 10.2.0.0/16
---
apiVersion: operator.tigera.io/v1
kind: APIServer
metadata:
  name: default
spec: {}
```

Le paramètre `nodeAddressAutodetectionV4.cidrs` oblige Calico à utiliser l'interface du réseau 10.2.0.0/16 pour faire communiquer les nœuds, et non une éventuelle interface NAT. Sans lui, sur une VM à deux cartes, les pods de deux nœuds différents ne pourraient pas se parler.

VM 1

```
kubectl create -f ~/calico-custom-resources.yaml
kubectl get pods -n calico-system -w
```

```
qassim@kubeqassim1:~$ kubectl get pods -n calico-system -w
NAME                                       READY   STATUS              RESTARTS
   AGE
calico-kube-controllers-69dcb444db-kkn8t   0/1     Pending             0
   24s
calico-node-9pkjs                          0/1     PodInitializing     0
   24s
calico-typha-6ccbbbf965-7jt54              1/1     Running             0
   24s
csi-node-driver-92pld                      0/2     ContainerCreating   0
   24s
calico-kube-controllers-69dcb444db-kkn8t   0/1     Pending             0
   24s
calico-kube-controllers-69dcb444db-kkn8t   0/1     ContainerCreating   0
   24s
calico-node-9pkjs                          0/1     Running             0
   40s
csi-node-driver-92pld                      0/2     ContainerCreating   0
   52s
calico-kube-controllers-69dcb444db-kkn8t   0/1     ContainerCreating   0
   53s
calico-kube-controllers-69dcb444db-kkn8t   0/1     Running             0
   61s
calico-kube-controllers-69dcb444db-kkn8t   1/1     Running             0
   61s
csi-node-driver-92pld                      2/2     Running             0
   66s
calico-node-9pkjs                          1/1     Running             0
   70s
```

*Figure 30 – Déploiement des pods Calico suivi en direct*

En un peu plus d'une minute, les pods passent de `Pending` à `PodInitializing` ou `ContainerCreating` (téléchargement des images, installation des binaires CNI dans `/opt/cni/bin`), puis à `Running`.

```
qassim@kubeqassim1:~$ kubectl get pods -n calico-systemem
kubectl get nodes
kubectl get pods -n kube-system -l k8s-app=kube-dns -o wide
kubectl get ippools -o wide 2>/dev/null || kubectl get ippool.crd.projectcalico
.org
NAME                                       READY   STATUS    RESTARTS   AGE
calico-kube-controllers-69dcb444db-kkn8t   1/1     Running   0          3m49s
calico-node-9pkjs                          1/1     Running   0          3m49s
calico-typha-6ccbbbf965-7jt54              1/1     Running   0          3m49s
csi-node-driver-92pld                      2/2     Running   0          3m49s
NAME     STATUS   ROLES           AGE     VERSION
k8s-cp   Ready    control-plane   8m25s   v1.34.12
NAME                       READY   STATUS    RESTARTS   AGE     IP
 NODE     NOMINATED NODE   READINESS GATES
coredns-66bc5c9577-8c2gn   1/1     Running   0          8m16s   10.244.62.130
 k8s-cp   <none>           <none>
coredns-66bc5c9577-8ms7g   1/1     Running   0          8m16s   10.244.62.132
 k8s-cp   <none>           <none>
NAME                  AGE
default-ipv4-ippool   3m53s
qassim@kubeqassim1:~$
```

*Figure 31 – Nœud Ready et CoreDNS Running avec des IP en 10.244.x.x*

Dès que le CNI est en place, la condition réseau du nœud bascule : `k8s-cp` passe **Ready**, et CoreDNS sort de Pending avec les adresses 10.244.62.130 et .132, prises dans le pool Calico. Chaque nœud reçoit un bloc /26 du pool, ici 10.244.62.128/26 pour le control plane.

# Partie 4 – Joindre le worker

VM 2

```
sudo kubeadm join 10.2.100.225:6443 --token <jeton> \
  --discovery-token-ca-cert-hash sha256:<empreinte-de-la-CA>
```

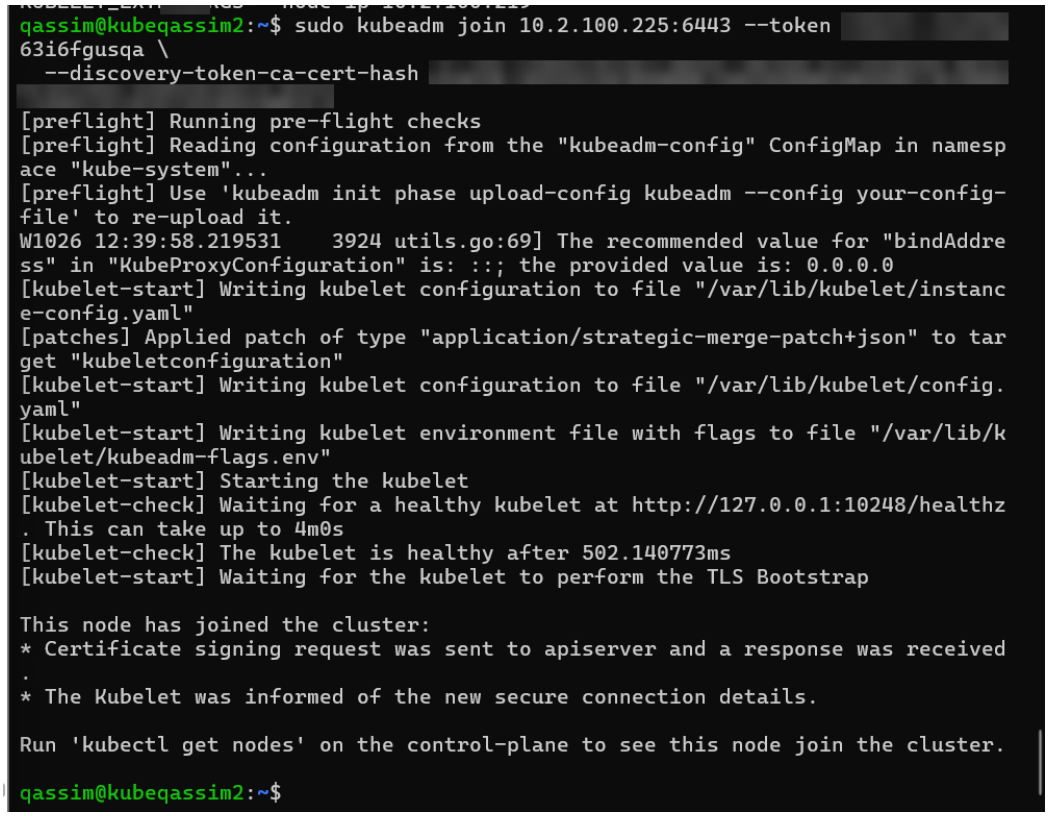



*Figure 32 – Le worker rejoint le cluster*

Le join est rapide et instructif. Le worker lit d'abord la configuration du cluster dans la ConfigMap `kubeadm-config`, puis écrit enfin les fichiers qui manquaient à son kubelet (`config.yaml`, `kubeadm-flags.env`). Le kubelet démarre, s'authentifie avec le jeton et envoie une demande de certificat que l'API signe automatiquement : c'est le **TLS Bootstrap**. L'empreinte `sha256` permet au worker de vérifier qu'il parle bien au bon control plane et pas à un imposteur.

VM 1

```
kubectl get nodes -o wide
kubectl get pods -n calico-system -o wide
```

```
qassim@kubeqassim1:~$ kubectl get nodes -o wide
kubectl get pods -n calico-system -o wide
NAME     STATUS   ROLES           AGE   VERSION    INTERNAL-IP    EXTERNAL-IP
 OS-IMAGE             KERNEL-VERSION     CONTAINER-RUNTIME
k8s-cp   Ready    control-plane   10m   v1.34.12   10.2.100.225   <none>
 Ubuntu 26.04.1 LTS   7.0.0-38-generic   containerd://2.2.2
k8s-w1   Ready    <none>          74s   v1.34.12   10.2.100.219   <none>
 Ubuntu 26.04.1 LTS   7.0.0-38-generic   containerd://2.2.2
NAME                                       READY   STATUS    RESTARTS   AGE
 IP              NODE     NOMINATED NODE   READINESS GATES
calico-kube-controllers-69dcb444db-kkn8t   1/1     Running   0          5m45s
 10.244.62.131   k8s-cp   <none>           <none>
calico-node-9pkjs                          1/1     Running   0          5m45s
 10.2.100.225    k8s-cp   <none>           <none>
calico-node-qspv7                          1/1     Running   0          74s
 10.2.100.219    k8s-w1   <none>           <none>
calico-typha-6ccbbbf965-7jt54              1/1     Running   0          5m45s
 10.2.100.225    k8s-cp   <none>           <none>
csi-node-driver-92pld                      2/2     Running   0          5m45s
 10.244.62.129   k8s-cp   <none>           <none>
csi-node-driver-wx2zh                      2/2     Running   0          74s
 10.244.228.65   k8s-w1   <none>           <none>
qassim@kubeqassim1:~$
```

*Figure 33 – Deux nœuds Ready et pods Calico déployés sur le worker*

**Le worker apparaît-il, avec quelle INTERNAL-IP ?** Oui : `k8s-w1` est **Ready** avec l'INTERNAL-IP **10.2.100.219**, celle imposée par `--node-ip`. Calico y a automatiquement déployé un `calico-node` et un `csi-node-driver`, car ce sont des DaemonSets (un pod par nœud). Le worker a reçu son propre bloc d'adresses de pods, 10.244.228.64/26.

VM 1

```
kubectl label node k8s-w1 node-role.kubernetes.io/worker=
kubectl get nodes
kubectl describe nodes | grep -E "^Name:|Taints:"
```

```
qassim@kubeqassim1:~$ kubectl label node k8s-w1 node-role.kubernetes.io/worker=

kubectl get nodes
node/k8s-w1 labeled
NAME     STATUS   ROLES           AGE    VERSION
k8s-cp   Ready    control-plane   10m    v1.34.12
k8s-w1   Ready    worker          107s   v1.34.12
qassim@kubeqassim1:~$ kubectl describe nodes | grep -E "^Name:|Taints:"
Name:               k8s-cp
Taints:             node-role.kubernetes.io/control-plane:NoSchedule
Name:               k8s-w1
Taints:             <none>
qassim@kubeqassim1:~$
```

*Figure 34 – Rôle worker affiché et taints des deux nœuds*

**Où un pod applicatif sera-t-il planifié par défaut ?** Sur `k8s-w1`. Le control plane porte le taint `node-role.kubernetes.io/control-plane:NoSchedule`, qui interdit au scheduler d'y placer un pod qui ne le tolère pas ; le worker n'a aucun taint. Le label `worker` est purement cosmétique : il remplit la colonne ROLES sans influencer le placement. C'est d'ailleurs une différence notable avec K3s au TP1, qui ne posait aucun taint sur son nœud unique.

**Résultat attendu atteint :** deux nœuds Ready, tous les pods de `kube-system` et de `calico-system` en Running.

## Fusion des kubeconfig (non réalisée)

L'énoncé demande de fusionner le kubeconfig de ce cluster avec celui du K3s du TP1 et de renommer les contextes `k3s` et `kubeadm`. J'avais supprimé la VM K3s pour libérer des ressources sur l'ESXi : cette étape n'a donc pas pu être faite. La procédure aurait été la suivante :

```
scp qassim@10.2.100.225:.kube/config ~/.kube/kubeadm.yaml
scp qassim@<IP_K3S>:.kube/config ~/.kube/k3s.yaml   # remplacer 127.0.0.1 par
l'IP de la VM K3s
KUBECONFIG=~/.kube/kubeadm.yaml:~/.kube/k3s.yaml kubectl config view --flatten >
~/.kube/config
kubectl config rename-context kubernetes-admin@kubernetes kubeadm
kubectl config rename-context default k3s
kubectl config get-contexts
kubectl config use-context kubeadm
```

Le piège à connaître : le kubeconfig de K3s pointe sur `https://127.0.0.1:6443`. Il faut remplacer cette adresse par l'IP de la VM avant de l'utiliser depuis une autre machine.

# Partie 5 – Premiers pods sur le nouveau cluster

VM 1

```
kubectl create namespace tp2
kubectl config set-context --current --namespace=tp2
```

## 1. Un pod simple : hello

~/pod-worker.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: hello
  labels:
    app: hello
spec:
  containers:
  - name: nginx
    image: nginx:1.27-alpine
    ports:
    - containerPort: 80
```

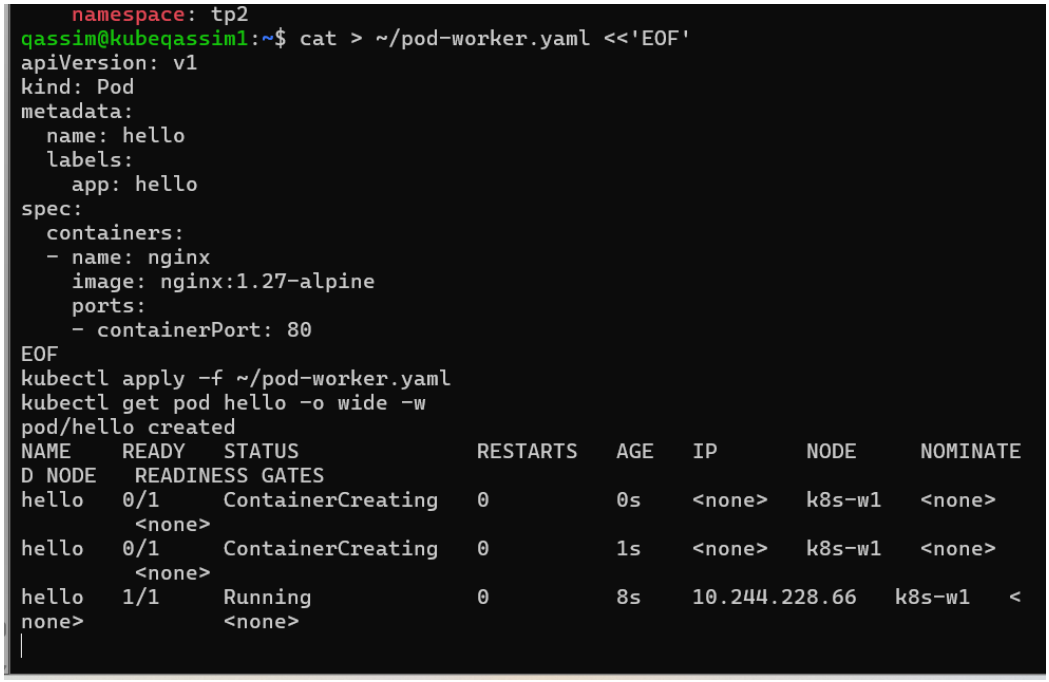



*Figure 35 – Le pod hello démarre sur k8s-w1*

> **Sur quel nœud atterrit-il et pourquoi ?** Sur `k8s-w1` (IP 10.244.228.66). Le pod ne déclare aucune toleration : le scheduler écarte donc le control plane à cause de son taint NoSchedule, et le worker reste le seul candidat.

## 2. Un pod multi-conteneurs : web-logs

Ce pod combine les trois types de conteneurs : un init container qui prépare un fichier, un serveur web qui le publie, et un « watcher » qui le relit toutes les 30 secondes. Tous partagent un volume `emptyDir`.

**~/pod-multi.yaml (extrait)**

```
apiVersion: v1
kind: Pod
metadata:
  name: web-logs
spec:
  volumes:
  - name: html
    emptyDir: {}
  initContainers:
  - name: prepare
    image: busybox:1.36
    command: ["sh", "-c", "echo \"Préparé par l'init container sur $(hostname) à
$(date)\" > /work/index.html"]
    volumeMounts:
    - { name: html, mountPath: /work }
  containers:
  - name: web
    image: nginx:1.27-alpine
    volumeMounts:
    - { name: html, mountPath: /usr/share/nginx/html }
  - name: watcher
    image: busybox:1.36
    command: ["sh", "-c", "while true; do cat /data/index.html; sleep 30; done"]
    volumeMounts:
    - { name: html, mountPath: /data }
```

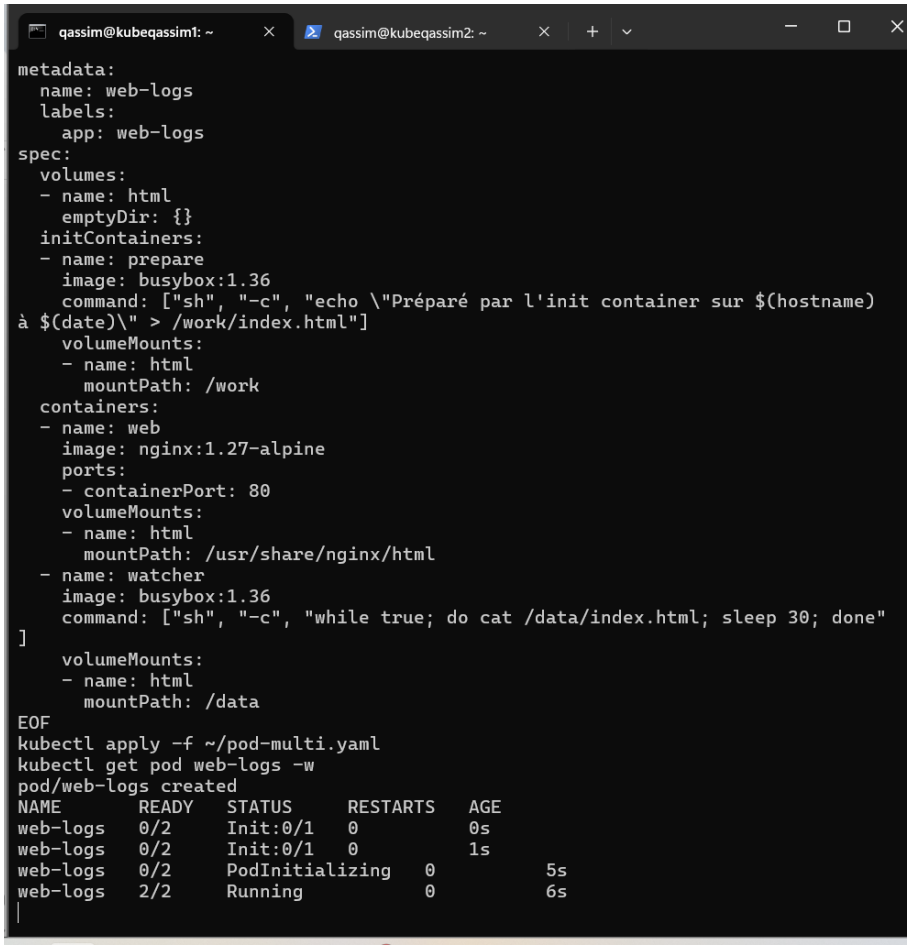



*Figure 36 – Progression Init:0/1 → PodInitializing → Running 2/2*

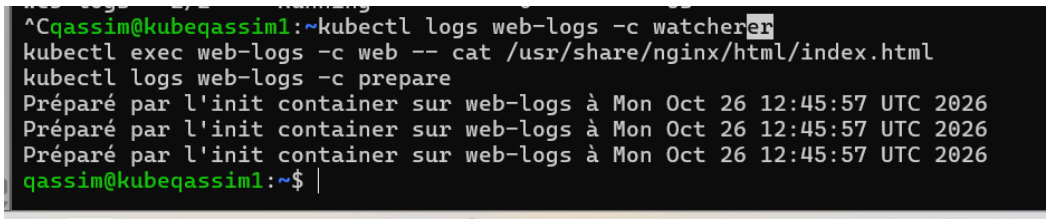



*Figure 37 – Le même fichier vu par le watcher et par le conteneur web*

Le fichier écrit par l'init container est bien lu par les deux autres conteneurs, chacun à travers son propre point de montage (`/work`, `/usr/share/nginx/html`, `/data`). Le hostname vaut `web-logs` : dans un pod, tous les conteneurs partagent le même nom d'hôte et la même IP. Le volume emptyDir naît et meurt avec le pod, c'est un espace de travail partagé et temporaire.

Pour répondre à la question sur l'échec de l'init container, j'ai créé une variante qui se termine par `exit 1` :

**VM 1**

```
sed -e 's/name: web-logs/name: web-logs-ko/' \
    -e 's|> /work/index.html"\]|> /work/index.html; exit 1"]|' \
    ~/pod-multi.yaml > ~/pod-multi-ko.yaml
kubectl apply -f ~/pod-multi-ko.yaml
sleep 40
kubectl get pod web-logs-ko
kubectl describe pod web-logs-ko | tail -8
```

```
qassim@kubeqassim1:~$ sed -e 's/name: web-logs/name: web-logs-ko/' \
    -e 's|> /work/index.html"\]|> /work/index.html; exit 1"]|' \
    ~/pod-multi.yaml > ~/pod-multi-ko.yaml
grep -n "exit 1" ~/pod-multi-ko.yaml
kubectl apply -f ~/pod-multi-ko.yaml
sleep 40
kubectl get pod web-logs-ko
kubectl describe pod web-logs-ko | tail -8
14:    command: ["sh", "-c", "echo \"Préparé par l'init container sur $(hostnam
e) à $(date)\" > /work/index.html; exit 1"]
pod/web-logs-ko created
NAME          READY   STATUS       RESTARTS      AGE
web-logs-ko   0/2     Init:Error   3 (23s ago)   41s
Events:
  Type     Reason     Age               From               Message
  ----     ------     ----              ----               -------
  Normal   Scheduled  41s               default-scheduler  Successfully assigne
d tp2/web-logs-ko to k8s-w1
  Normal   Pulled     1s (x4 over 38s)  kubelet            Container image "bus
ybox:1.36" already present on machine
  Normal   Created    1s (x4 over 38s)  kubelet            Created container: p
repare
  Normal   Started    1s (x4 over 38s)  kubelet            Started container pr
epare
  Warning  BackOff    1s (x5 over 36s)  kubelet            Back-off restarting
failed container prepare in pod web-logs-ko_tp2(8b3cac8b-91ff-4477-946c-5e536f0
7f801)
qassim@kubeqassim1:~$
```

*Figure 38 – Init container en échec : le pod reste en Init:Error*

**Que se passe-t-il si l'init container échoue ?** Le pod reste bloqué en `Init:Error` puis `Init:CrashLoopBackOff` (0/2, trois redémarrages en 41 secondes). Les conteneurs `web` et `watcher` ne démarrent jamais : Kubernetes exige que tous les init containers se terminent avec succès, dans l'ordre, avant de lancer le reste du pod. L'init container est relancé selon la `restartPolicy` avec un délai qui double à chaque échec.

## 3. Un pod sur le control plane : sur-le-cp

**~/pod-toleration.yaml**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sur-le-cp
spec:
  nodeSelector:
    node-role.kubernetes.io/control-plane: ""
  tolerations:
  - key: node-role.kubernetes.io/control-plane
    operator: Exists
    effect: NoSchedule
  containers:
  - name: busybox
    image: busybox:1.36
    command: ["sleep", "3600"]
```

```
qassim@kubeqassim1:~$ cat > ~/pod-toleration.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: sur-le-cp
spec:
  nodeSelector:
    node-role.kubernetes.io/control-plane: ""
  tolerations:
  - key: node-role.kubernetes.io/control-plane
    operator: Exists
    effect: NoSchedule
  containers:
  - name: busybox
    image: busybox:1.36
    command: ["sleep", "3600"]
EOF
kubectl apply -f ~/pod-toleration.yaml
kubectl get pod sur-le-cp -o wide -w
pod/sur-le-cp created
NAME        READY   STATUS              RESTARTS   AGE   IP       NODE     NOMI
NATED NODE   READINESS GATES
sur-le-cp   0/1     ContainerCreating   0          0s    <none>   k8s-cp   <non
e>           <none>
sur-le-cp   0/1     ContainerCreating   0          1s    <none>   k8s-cp   <non
e>           <none>
sur-le-cp   1/1     Running             0          4s    10.244.62.135   k8s-cp
   <none>           <none>
^Cqassim@kubeqassim1:~$
```

*Figure 39 – Le pod sur-le-cp tourne bien sur k8s-cp*

> **Pourquoi faut-il une toleration ET un nodeSelector ?** Les deux n'ont pas le même rôle. La **toleration** lève l'interdiction posée par le taint : elle **autorise** le pod à aller sur le control plane, mais ne l'y **oblige** pas. Avec elle seule, le scheduler resterait libre de choisir le worker. Le **nodeSelector** impose, lui, un nœud portant le label control-plane. Il faut donc lever l'interdiction et désigner la cible.

## 4. Connectivité entre les deux nœuds

VM 1

```
HELLO_IP=$(kubectl get pod hello -o jsonpath='{.status.podIP}')
kubectl exec sur-le-cp -- wget -qO- http://$HELLO_IP | head -5
ip -d link show vxlan.calico
```

```
^Cqassim@kubeqassim1:~kubectl get pods -o widede
HELLO_IP=$(kubectl get pod hello -o jsonpath='{.status.podIP}')
echo $HELLO_IP
kubectl exec sur-le-cp -- wget -qO- http://$HELLO_IP | head -5
ip -d link show vxlan.calico
NAME        READY   STATUS    RESTARTS   AGE     IP              NODE     NOMIN
ATED NODE   READINESS GATES
hello       1/1     Running   0          5m58s   10.244.228.66   k8s-w1   <none
>           <none>
sur-le-cp   1/1     Running   0          69s     10.244.62.135   k8s-cp   <none
>           <none>
web-logs    2/2     Running   0          4m26s   10.244.228.67   k8s-w1   <none
>           <none>
10.244.228.66
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
3: vxlan.calico: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state
 UNKNOWN mode DEFAULT group default qlen 1000
    link/ether 66:aa:e5:3f:6c:75 brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0
 minmtu 68 maxmtu 65535
    vxlan id 4096 local 10.2.100.225 dev ens192 srcport 0 0 dstport 4789 ttl au
to ageing 300 reserved_bits 0xf7ffffff000000ff nolearning addrgenmode eui64 num
txqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 655
36 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_max_s
ize 65536
qassim@kubeqassim1:~$
```

*Figure 40 – Le pod du control plane récupère la page nginx du pod du worker*

**VM 2**

```
ip -d link show vxlan.calico
ip route | grep 10.244
```

```
qassim@kubeqassim2:~$ ip -d link show vxlan.calico
ip route | grep 10.244
4: vxlan.calico: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state
 UNKNOWN mode DEFAULT group default qlen 1000
    link/ether 66:74:91:32:7c:04 brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0
 minmtu 68 maxmtu 65535
    vxlan id 4096 local 10.2.100.219 dev ens192 srcport 0 0 dstport 4789 ttl au
to ageing 300 reserved_bits 0xf7ffffff000000ff nolearning addrgenmode eui64 num
txqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 655
36 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_max_s
ize 65536
10.244.62.128 via 10.2.100.225 dev ens192 proto 80 onlink
10.244.62.128/26 via 10.2.100.225 dev ens192 proto 80 onlink
blackhole 10.244.228.64/26 proto 80
10.244.228.65 dev calib33d087aaec scope link
10.244.228.66 dev cali5cc7963977f scope link
10.244.228.67 dev cali47602aa592b scope link
qassim@kubeqassim2:~$
```

*Figure 41 – Interface vxlan.calico et table de routage du worker*

La communication fonctionne : `sur-le-cp` (10.244.62.135, sur k8s-cp) obtient la page « Welcome to nginx! » de `hello` (10.244.228.66, sur k8s-w1). L'interface `vxlan.calico` existe bien sur les deux nœuds (VNI 4096, port UDP 4789, MTU 1450).

En regardant la table de routage du worker, j'ai toutefois remarqué un détail qui contredit l'intuition : la route vers les pods du control plane est `10.244.62.128/26 via 10.2.100.225 dev ens192`. Elle passe directement par la carte physique, pas par l'interface VXLAN. Pour en avoir le cœur net, j'ai capturé le trafic VXLAN sur le worker pendant une requête :

```
sudo timeout 15 tcpdump -ni ens192 udp port 4789 &     # VM 2
kubectl exec sur-le-cp -- wget -qO- http://$HELLO_IP > /dev/null   # VM 1
```

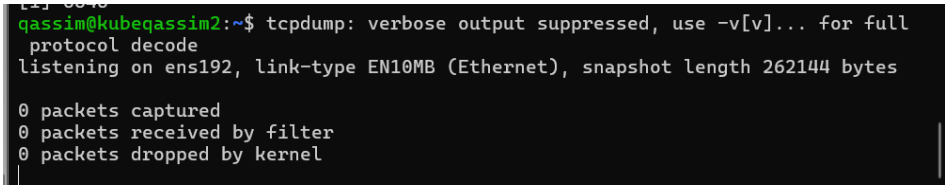



*Figure 42 – Aucun paquet VXLAN capturé pendant l'échange*

**Le trafic traverse-t-il le VXLAN Calico ?** Non, et c'est voulu. Le mode `VXLANCrossSubnet` n'encapsule les paquets que lorsque les nœuds se trouvent dans des sous-réseaux différents. Mes deux nœuds partagent le même réseau 10.2.0.0/16 : Calico route donc les paquets des pods nativement, ce qui évite le coût de l'encapsulation (en-têtes en plus, MTU réduite). Le tunnel reste prêt pour d'éventuels nœuds situés dans un autre sous-réseau. Le reste de la table montre une interface `cali…` par pod local et une route `blackhole` sur le bloc /26 du nœud, qui rejette le trafic vers des adresses non attribuées.

## Partie 6 – Clinique des pods

Les manifests cassés étaient fournis dans l'archive `TP2.zip` de la plateforme. Celle-ci l'avait enregistrée sous le nom `TP2.zip.txt`, ce qui m'a valu quelques erreurs `No such file` avant de comprendre. Une fois le bon nom identifié, je l'ai envoyée sur le control plane avec `scp` puis décompressée.

**PowerShell (PC)**

```
scp "$env:USERPROFILE\Downloads\TP2.zip.txt" qassim@10.2.100.225:~/TP2.zip
```

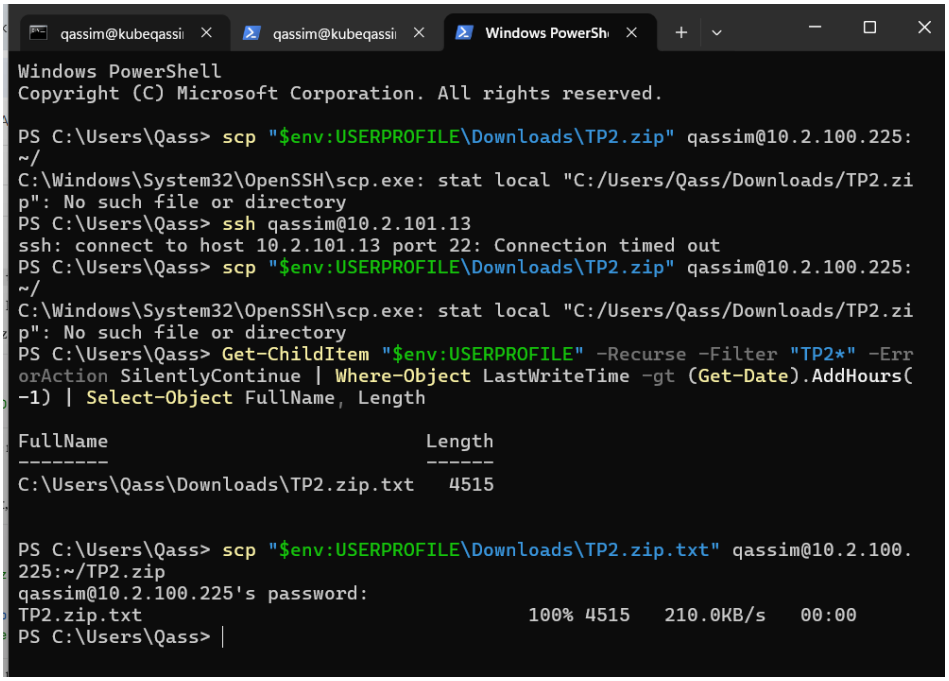



*Figure 43 – Transfert de l'archive vers k8s-cp*

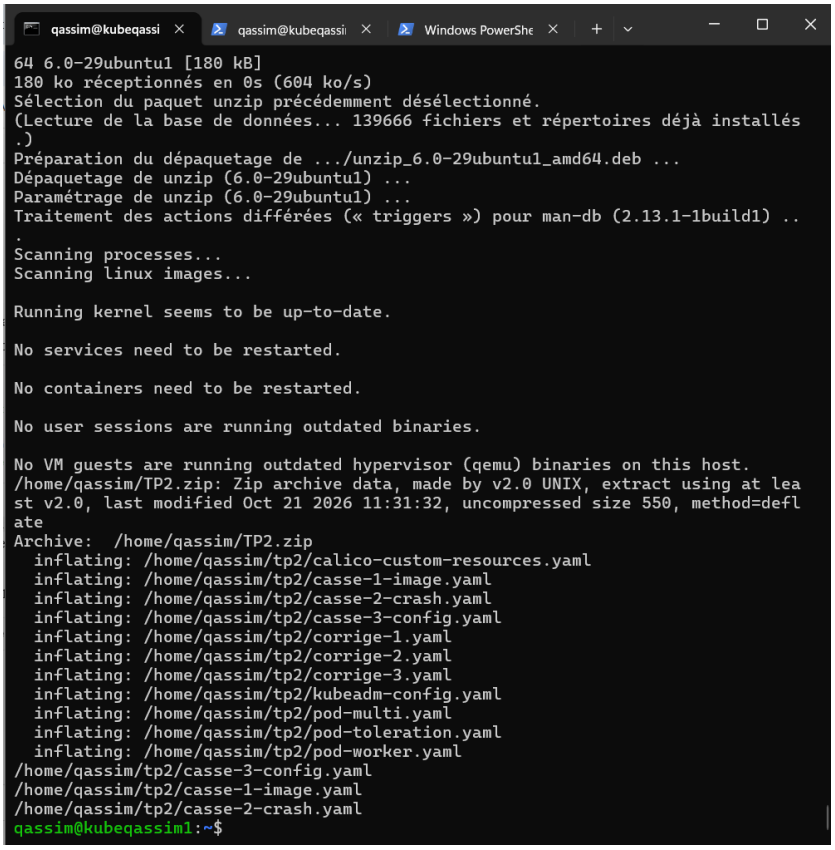



*Figure 44 – Décompression de l'archive : les trois manifests cassés*

Pour respecter la consigne de ne pas consulter les corrigés avant d'avoir posé mon diagnostic, j'ai écrit mes propres corrections dans un dossier séparé, `~/tp2/mes-corrections/`. Les fichiers fournis n'ont ainsi été ni ouverts ni écrasés.

VM 1

```
cd ~/tp2
kubectl apply -f casse-1-image.yaml -f casse-2-crash.yaml -f casse-3-config.yaml
sleep 60
kubectl get pods
```

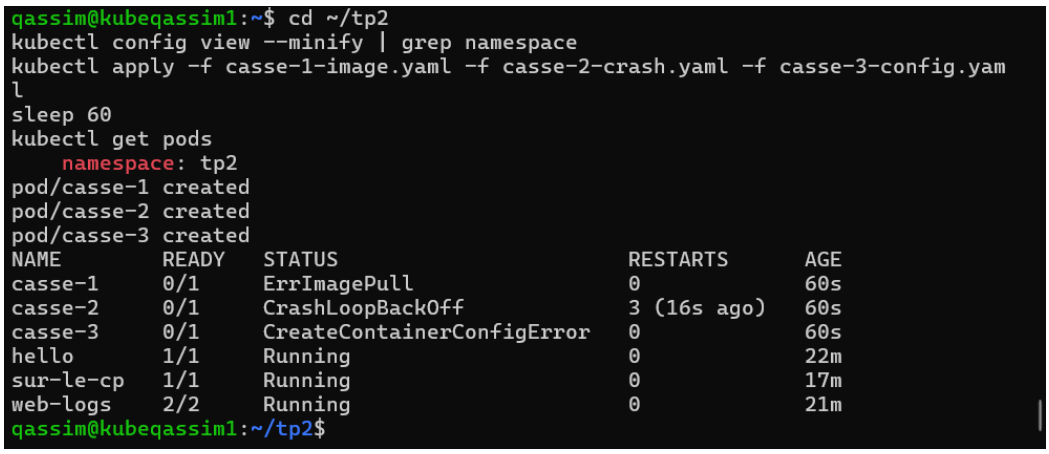



*Figure 45 – État initial des trois pods cassés*

Les trois pods sont cassés de trois façons bien différentes. Pour chacun, j'ai suivi la même démarche : `describe` et ses Events, puis `logs` (et `logs --previous`), puis lecture du manifest si nécessaire.

## Cas 1 – ErrImagePull / ImagePullBackOff

VM 1

```
kubectl describe pod casse-1 | grep -E "Image:|State:|Reason:"
kubectl describe pod casse-1 | tail -10
kubectl logs casse-1
```

```
qassim@kubeqassim1:~/tp2$ kubectl describe pod casse-1 | grep -E "Image:|State:
|Reason:"
kubectl describe pod casse-1 | tail -10
kubectl logs casse-1
    Image:          nginx:1.27-alpin
    State:          Waiting
      Reason:       ImagePullBackOff
                    node.kubernetes.io/unreachable:NoExecute op=Exists
 for 300s
Events:
  Type     Reason     Age                  From               Message
  ----     ------     ----                 ----               -------
  Normal   Scheduled  2m2s                 default-scheduler  Successfully assig
ned tp2/casse-1 to k8s-w1
  Normal   Pulling    30s (x4 over 2m2s)   kubelet            Pulling image "ngi
nx:1.27-alpin"
  Warning  Failed     29s (x4 over 2m1s)   kubelet            Failed to pull ima
ge "nginx:1.27-alpin": rpc error: code = NotFound desc = failed to pull and unp
ack image "docker.io/library/nginx:1.27-alpin": failed to resolve image: docker
.io/library/nginx:1.27-alpin: not found
  Warning  Failed     29s (x4 over 2m1s)   kubelet            Error: ErrImagePul
l
  Normal   BackOff    4s (x7 over 2m)      kubelet            Back-off pulling i
mage "nginx:1.27-alpin"
  Warning  Failed     4s (x7 over 2m)      kubelet            Error: ImagePullBa
ckOff
Error from server (BadRequest): container "web" in pod "casse-1" is waiting to
start: trying and failing to pull image
qassim@kubeqassim1:~/tp2$
```

*Figure 46 – Diagnostic du cas 1 : image introuvable*

| | Diagnostic |
|---|---|
| **Symptôme** | Pod en 0/1 qui alterne entre `ErrImagePull` et `ImagePullBackOff`, avec un délai croissant entre les tentatives. Aucun log : le conteneur n'a jamais existé. |
| **Cause** | Le tag `nginx:1.27-alpin` n'existe pas sur Docker Hub (« not found ») : faute de frappe, il manque le « e » de `alpine`. |
| **Correction** | Corriger le tag en `nginx:1.27-alpine`. Le champ `image` est modifiable sur un pod existant : un simple `kubectl apply` suffit. |

VM 1

```
mkdir -p ~/tp2/mes-corrections
sed 's/nginx:1.27-alpin$/nginx:1.27-alpine/' ~/tp2/casse-1-image.yaml >
~/tp2/mes-corrections/corrige-1.yaml
diff ~/tp2/casse-1-image.yaml ~/tp2/mes-corrections/corrige-1.yaml
kubectl apply -f ~/tp2/mes-corrections/corrige-1.yaml
```

```
qassim@kubeqassim1:~/tp2$ mkdir -p ~/tp2/mes-corrections
sed 's/nginx:1.27-alpin$/nginx:1.27-alpine/' ~/tp2/casse-1-image.yaml > ~/tp2/m
es-corrections/corrige-1.yaml
diff ~/tp2/casse-1-image.yaml ~/tp2/mes-corrections/corrige-1.yaml
kubectl apply -f ~/tp2/mes-corrections/corrige-1.yaml
sleep 15
kubectl get pod casse-1
9c9
<       image: nginx:1.27-alpin
---
>       image: nginx:1.27-alpine
pod/casse-1 configured
NAME      READY   STATUS    RESTARTS   AGE
casse-1   1/1     Running   0          2m51s
qassim@kubeqassim1:~/tp2$
```

*Figure 47 – Cas 1 corrigé en place : pod/casse-1 configured, Running*

## Cas 2 – CrashLoopBackOff

VM 1

```
kubectl logs casse-2
kubectl logs casse-2 --previous
kubectl describe pod casse-2 | grep -A5 "Last State"
cat -n ~/tp2/casse-2-crash.yaml
```

```
qassim@kubeqassim1:~/tp2$ kubectl get pod casse-2
kubectl logs casse-2
kubectl logs casse-2 --previous
kubectl describe pod casse-2 | grep -A5 "Last State"
NAME      READY   STATUS             RESTARTS      AGE
casse-2   0/1     CrashLoopBackOff   5 (14s ago)   3m12s
ERREUR : variable DB_HOST absente
Démarrage de l application
ERREUR : variable DB_HOST absente
Démarrage de l application
    Last State:     Terminated
      Reason:       Error
      Exit Code:    1
      Started:      Mon, 26 Oct 2026 13:09:01 +0000
      Finished:     Mon, 26 Oct 2026 13:09:01 +0000
    Ready:          False
qassim@kubeqassim1:~/tp2$
```

*Figure 48 – Diagnostic du cas 2 : sortie en erreur, code 1*

```
qassim@kubeqassim1:~/tp2$ cat -n ~/tp2/casse-2-crash.yaml
     1  apiVersion: v1
     2  kind: Pod
     3  metadata:
     4    name: casse-2
     5    namespace: tp2
     6  spec:
     7    containers:
     8      - name: app
     9        image: busybox:1.37
    10        command: ["sh", "-c", "echo 'Démarrage de l application' ; echo '
ERREUR : variable DB_HOST absente' >&2 ; exit 1"]
qassim@kubeqassim1:~/tp2$
```

*Figure 49 – Le manifest révèle une sortie en erreur inconditionnelle*

| | Diagnostic |
|---|---|
| **Symptôme** | Pod en `CrashLoopBackOff`, 5 redémarrages en 3 minutes. `Last State: Terminated`, `Reason: Error`, `Exit Code: 1`. Le kubelet relance le conteneur avec un délai qui double (10 s, 20 s… jusqu'à 5 min). |

| | Diagnostic |
|---|---|
| **Cause** | Les logs (y compris `--previous`, ceux de l'instance qui a planté) affichent `ERREUR : variable DB_HOST absente`. Le programme exige une variable `DB_HOST` qui n'est pas fournie et s'arrête en erreur. En lisant le manifest, j'ai vu qu'il sortait même en erreur sans tester la variable. |
| **Correction** | Fournir `DB_HOST` dans `env` et faire tester la variable par le programme, qui continue ensuite avec `sleep`. `command` et `env` ne sont pas modifiables : il faut supprimer puis recréer le pod. |

**~/tp2/mes-corrections/corrige-2.yaml**

```
apiVersion: v1
kind: Pod
metadata:
  name: casse-2
  namespace: tp2
spec:
  containers:
    - name: app
      image: busybox:1.37
      env:
        - name: DB_HOST
          value: "db"
      command: ["sh", "-c", "echo 'Démarrage de l application' ; if [ -
z \"$DB_HOST\" ]; then echo 'ERREUR : variable DB_HOST absente' >&2 ; exit 1 ; fi
; echo \"Connexion à $DB_HOST\" ; sleep 3600"]
```

J'ai volontairement tenté un `apply` direct avant de supprimer le pod, pour vérifier la règle : l'API l'a refusé avec `pod updates may not change fields other than…`, en montrant que `Command` et `Env` étaient concernés.

**VM 1**

```
kubectl delete pod casse-2
kubectl apply -f ~/tp2/mes-corrections/corrige-2.yaml
kubectl get pod casse-2
kubectl logs casse-2
```

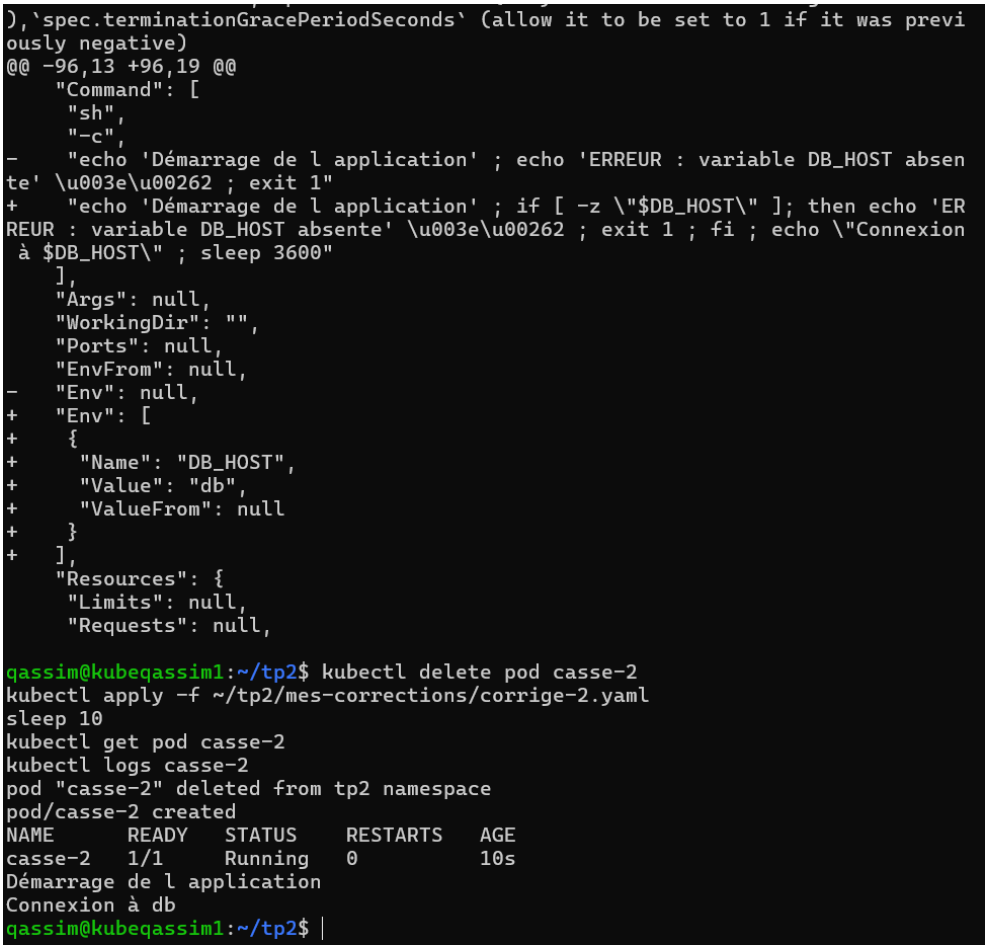



*Figure 50 – Refus de modification, puis recréation : casse-2 Running*

## Cas 3 – CreateContainerConfigError

**VM 1**

```
kubectl describe pod casse-3 | tail -6
kubectl logs casse-3
cat -n ~/tp2/casse-3-config.yaml
kubectl get configmap
```

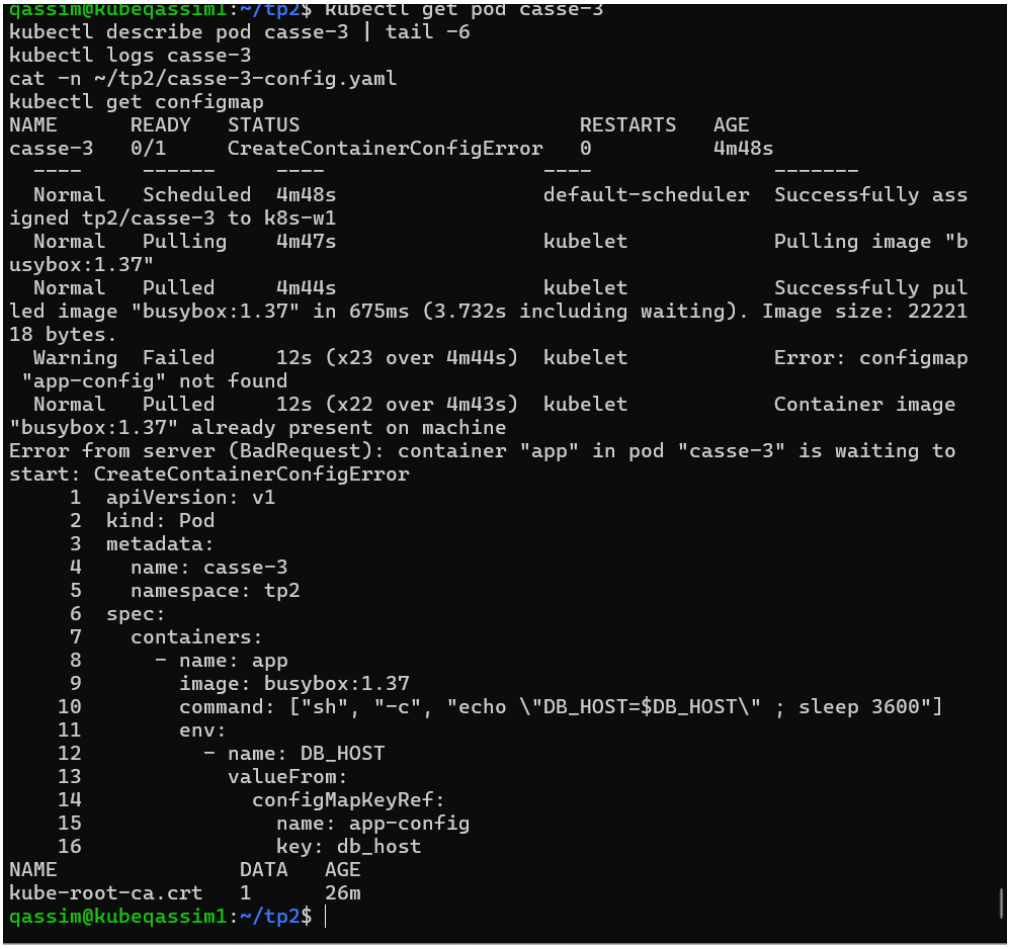



*Figure 51 – Diagnostic du cas 3 : ConfigMap introuvable*

| | Diagnostic |
|---|---|
| **Symptôme** | Pod en `CreateContainerConfigError`, sans aucun redémarrage et sans logs, alors que l'image a bien été téléchargée. L'événement `configmap "app-config" not found` se répète (23 fois en moins de 5 minutes). |
| **Cause** | La variable `DB_HOST` est lue dans la clé `db_host` d'une ConfigMap `app-config` qui n'existe pas dans le namespace `tp2` (seule `kube-root-ca.crt` est présente). Le kubelet ne peut pas construire l'environnement : le conteneur n'est jamais créé. |
| **Correction** | Créer la ConfigMap `app-config` avec la clé `db_host`. Inutile de recréer le pod : le kubelet réessaie en boucle et démarre le conteneur dès que la ConfigMap existe. |

**~/tp2/mes-corrections/corrige-3.yaml**

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: tp2
data:
  db_host: "db"
```

```
qassim@kubeqassim1:~/tp2$ cat > ~/tp2/mes-corrections/corrige-3.yaml <<'EOF'
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: tp2
data:
  db_host: "db"
EOF
kubectl apply -f ~/tp2/mes-corrections/corrige-3.yaml
kubectl get configmap app-config
sleep 20
kubectl get pod casse-3
kubectl logs casse-3
configmap/app-config created
NAME         DATA   AGE
app-config   1      0s
NAME      READY   STATUS    RESTARTS   AGE
casse-3   1/1     Running   0          5m30s
DB_HOST=db
qassim@kubeqassim1:~/tp2$
```

*Figure 52 – ConfigMap créée : casse-3 démarre sans être recréé*

## Bilan de la clinique

```
qassim@kubeqassim1:~/tp2$ kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE     IP              NODE     NOMIN
ATED NODE   READINESS GATES
casse-1     1/1     Running   0          6m11s   10.244.228.69   k8s-w1   <none
>           <none>
casse-2     1/1     Running   0          114s    10.244.228.72   k8s-w1   <none
>           <none>
casse-3     1/1     Running   0          6m11s   10.244.228.71   k8s-w1   <none
>           <none>
hello       1/1     Running   0          27m     10.244.228.66   k8s-w1   <none
>           <none>
sur-le-cp   1/1     Running   0          23m     10.244.62.135   k8s-cp   <none
>           <none>
web-logs    2/2     Running   0          26m     10.244.228.67   k8s-w1   <none
>           <none>
qassim@kubeqassim1:~/tp2$
```

*Figure 53 – Les six pods du namespace tp2 en Running*

Ce qui me paraît le plus parlant, c'est que les trois corrections ne se font pas du tout de la même manière, et que l'âge des pods le montre : casse-1 a été modifié en place, casse-3 a simplement été débloqué, tandis que casse-2 est le seul à avoir été recréé (114 secondes d'âge et une nouvelle IP).

| Cas | État | Outil décisif | Type de correction |
|---|---|---|---|
| 1 | ImagePullBackOff | Events de `describe` | Modification en place (`apply`) |
| 2 | CrashLoopBackOff | `logs --previous` + Exit Code | Suppression et recréation |
| 3 | CreateContainerConfigError | Events de `describe` (pas de logs) | Création de la ressource manquante |

# Final – Jetons, maintenance et sauvegarde etcd

## Jetons de join

VM 1

```
kubeadm token list
kubeadm token create --ttl 1h --print-join-command
kubeadm token create --ttl 2m --description "test expiration"
kubeadm token list
```

Pour observer une expiration sans attendre une heure, j'ai créé en plus du jeton d'une heure demandé un jeton de deux minutes.

```
qassim@kubeqassim1:~/tp2$ kubeadm token list
TOKEN                     TTL         EXPIRES                USAGES
      DESCRIPTION                                                EXTRA GROUPS
0dx9nh.clodm6msql24cgmj   1m          2026-10-26T13:14:39Z   authentication,sig
ning   test expiration                                           system:bootst
rappers:kubeadm:default-node-token
egxx2i.m4v2pngwesh019e0   59m         2026-10-26T14:12:39Z   authentication,sig
ning   <none>                                                   system:bootst
rappers:kubeadm:default-node-token
[redacted]                23h         2026-10-27T12:30:54Z   authentication,sig
ning   The default bootstrap token generated by 'kubeadm init'.   system:bootst
rappers:kubeadm:default-node-token
qassim@kubeqassim1:~/tp2$ kubectl get pods -o wide
kubectl drain k8s-w1 --ignore-daemonsets --delete-emptydir-data
NAME        READY   STATUS    RESTARTS   AGE     IP              NODE     NOMIN
ATED NODE   READINESS GATES
casse-1     1/1     Running   0          6m51s   10.244.228.69   k8s-w1   <none
>           <none>
casse-2     1/1     Running   0          2m34s   10.244.228.72   k8s-w1   <none
>           <none>
casse-3     1/1     Running   0          6m51s   10.244.228.71   k8s-w1   <none
>           <none>
hello       1/1     Running   0          28m     10.244.228.66   k8s-w1   <none
>           <none>
sur-le-cp   1/1     Running   0          23m     10.244.62.135   k8s-cp   <none
>           <none>
web-logs    2/2     Running   0          27m     10.244.228.67   k8s-w1   <none
>           <none>
node/k8s-w1 cordoned
error: unable to drain node "k8s-w1" due to error: cannot delete cannot delete
Pods that declare no controller (use --force to override): tp2/casse-1, tp2/cas
se-2, tp2/casse-3, tp2/hello, tp2/web-logs, continuing command...
There are pending nodes to be drained:
 k8s-w1
cannot delete cannot delete Pods that declare no controller (use --force to ove
rride): tp2/casse-1, tp2/casse-2, tp2/casse-3, tp2/hello, tp2/web-logs
qassim@kubeqassim1:~/tp2$
```

*Figure 54 – Trois jetons (1 min, 59 min, 23 h), puis premier drain refusé*

Quelques minutes plus tard, le jeton de test avait disparu de la liste : le contrôleur *tokencleaner* du controller-manager supprime les jetons dès leur expiration. Tous ces jetons ont le même usage (`authentication,signing`) et le même groupe (`system:bootstrappers`) : ils ne servent qu'à faire entrer un nouveau nœud dans le cluster.

## Maintenance : drain et uncordon

Le premier `drain` a été refusé (voir la figure précédente) : `cannot delete Pods that declare no controller`. Le nœud a quand même été marqué non planifiable (`cordoned`). Kubernetes refuse par prudence de supprimer des pods qui ne seraient recréés par personne. J'ai donc ajouté `--force` :

VM 1

```
kubectl drain k8s-w1 --ignore-daemonsets --delete-emptydir-data --force
kubectl get nodes
kubectl get pods -o wide
```

```
qassim@kubeqassim1:~/tp2$ kubectl drain k8s-w1 --ignore-daemonsets --delete-emp
tydir-data --force
kubectl get nodes
kubectl get pods -o wide
node/k8s-w1 already cordoned
Warning: deleting Pods that declare no controller: tp2/casse-1, tp2/casse-2, tp
2/casse-3, tp2/hello, tp2/web-logs; ignoring DaemonSet-managed Pods: calico-sys
tem/calico-node-qspv7, calico-system/csi-node-driver-wx2zh, kube-system/kube-pr
oxy-scz7b
evicting pod tp2/web-logs
evicting pod tp2/casse-2
evicting pod tp2/casse-1
evicting pod tp2/casse-3
evicting pod tp2/hello
pod/casse-1 evicted
pod/hello evicted
pod/casse-2 evicted
pod/web-logs evicted
pod/casse-3 evicted
node/k8s-w1 drained
NAME     STATUS                     ROLES           AGE   VERSION
k8s-cp   Ready                      control-plane   43m   v1.34.12
k8s-w1   Ready,SchedulingDisabled   worker          34m   v1.34.12
NAME        READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINAT
ED NODE   READINESS GATES
sur-le-cp   1/1     Running   0          24m   10.244.62.135   k8s-cp   <none>
          <none>
qassim@kubeqassim1:~/tp2$
```

*Figure 55 – Drain forcé : cinq pods évincés, seul sur-le-cp subsiste*

**Que deviennent les pods ?** Les cinq pods de `k8s-w1` (casse-1, 2, 3, hello, web-logs) ont été évincés et **n'ont été recréés nulle part** : sans Deployment ni ReplicaSet, personne ne les relance. Seul `sur-le-cp`, qui tourne sur le control plane, a survécu. Les pods des DaemonSets (calico-node, csi-node-driver, kube-proxy) ont été ignorés grâce à `--ignore-daemonsets`, et `--delete-emptydir-data` était nécessaire à cause du volume de web-logs, dont les données sont perdues. Le nœud passe en `Ready,SchedulingDisabled`.

VM 1

```
kubectl uncordon k8s-w1
kubectl get nodes
kubeadm token list
```

```
qassim@kubeqassim1:~/tp2$ kubectl uncordon k8s-w1
kubectl get nodes
kubeadm token list
node/k8s-w1 already uncordoned
NAME     STATUS   ROLES           AGE   VERSION
k8s-cp   Ready    control-plane   44m   v1.34.12
k8s-w1   Ready    worker          35m   v1.34.12
TOKEN                     TTL         EXPIRES                USAGES
      DESCRIPTION                                            EXTRA GROUPS
egxx2i.m4v2pngwesh019e0   57m         2026-10-26T14:12:39Z   authentication,sig
ning   <none>                                                 system:bootst
rappers:kubeadm:default-node-token
[redacted]                23h         2026-10-27T12:30:54Z   authentication,sig
ning   The default bootstrap token generated by 'kubeadm init'.   system:bootst
rappers:kubeadm:default-node-token
qassim@kubeqassim1:~/tp2$
```

*Figure 56 – Nœud remis en service et jeton de test expiré*

> **Leçon retenue :** des pods créés à la main disparaissent à la première maintenance. C'est exactement pour cette raison que, dès le TP suivant, tout passera par des Deployments.

## Sauvegarde etcd (aperçu)

L'énoncé proposait de passer par `sh -c` dans le pod etcd, mais l'image etcd 3.6 est minimaliste et ne contient pas de shell. J'ai donc appelé `etcdctl` directement. La variable `ETCDCTL_API=3` n'est plus nécessaire, c'est la valeur par défaut depuis etcd 3.4.

VM 1

```
kubectl -n kube-system exec etcd-k8s-cp -- etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/lib/etcd/snapshot-tp2.db
sudo ls -lh /var/lib/etcd/snapshot-tp2.db
kubectl -n kube-system exec etcd-k8s-cp -- etcdutl snapshot status
/var/lib/etcd/snapshot-tp2.db -w table
```

```
qassim@kubeqassim1:~/tp2$ kubectl -n kube-system exec etcd-k8s-cp -- etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/lib/etcd/snapshot-tp2.db
sudo ls -lh /var/lib/etcd/snapshot-tp2.db
kubectl -n kube-system exec etcd-k8s-cp -- etcdutl snapshot status /var/lib/etc
d/snapshot-tp2.db -w table
{"level":"info","ts":"2026-10-26T13:14:49.314431Z","caller":"snapshot/v3_snapsh
ot.go:83","msg":"created temporary db file","path":"/var/lib/etcd/snapshot-tp2.
db.part"}
{"level":"info","ts":"2026-10-26T13:14:49.334893Z","logger":"client","caller":"
v3@v3.6.5/maintenance.go:236","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":"2026-10-26T13:14:49.354471Z","caller":"snapshot/v3_snapsh
ot.go:96","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":"2026-10-26T13:14:49.474175Z","logger":"client","caller":"
v3@v3.6.5/maintenance.go:302","msg":"completed snapshot read; closing"}
{"level":"info","ts":"2026-10-26T13:14:49.596414Z","caller":"snapshot/v3_snapsh
ot.go:111","msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","size":
"9.2 MB","took":"280.940584ms","etcd-version":"3.6.0"}
{"level":"info","ts":"2026-10-26T13:14:49.596634Z","caller":"snapshot/v3_snapsh
ot.go:121","msg":"saved","path":"/var/lib/etcd/snapshot-tp2.db"}
Snapshot saved at /var/lib/etcd/snapshot-tp2.db
Server version 3.6.0
-rw------- 1 root root 8.8M Oct 26 13:14 /var/lib/etcd/snapshot-tp2.db
+----------+----------+------------+------------+---------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE | VERSION |
+----------+----------+------------+------------+---------+
| 7db18289 |     6131 |        667 |     9.2 MB |   3.6.0 |
+----------+----------+------------+------------+---------+
qassim@kubeqassim1:~/tp2$
```

*Figure 57 – Snapshot etcd créé puis vérifié*

| Élément | Valeur |
|---|---|
| Fichier | `/var/lib/etcd/snapshot-tp2.db` – 8,8 Mo, permissions 600 |
| Version etcd | 3.6.0 |
| Hash / révision | `7db18289` / 6131 |
| Nombre de clés | 667 objets |
| Taille totale | 9,2 Mo |

etcd n'accepte que des connexions en TLS mutuel, d'où les trois certificats passés à `etcdctl`. Le fichier est visible directement sur la VM car `/var/lib/etcd` est un dossier de l'hôte monté dans le pod. `etcdctl snapshot save` crée la sauvegarde à travers le réseau, `etcdutl snapshot status` l'inspecte hors ligne. Cette procédure sera industrialisée au TP8.

## Difficultés rencontrées

| Problème | Cause | Solution |
|---|---|---|
| Assistant ESXi proposant 1 vCPU | Valeur par défaut | Passage à 2 vCPU, minimum exigé par kubeadm |
| Saisie impossible dans un champ de l'installateur | Focus et disposition clavier de la console web | Navigation à la touche Tab, clavier français dans la console |
| / limité à 18,5 Go | L'installateur n'utilise que la moitié du LVM | `lvextend -r -l +100%FREE` |
| IP en DHCP | Risque de changement d'IP qui casserait les certificats | IP statique via netplan |
| Variables présentes sur une seule VM | Une variable tapée dans chaque onglet | Ressaisie des deux variables sur chaque VM |
| kubelet `inactive (dead)` et non en boucle | Le paquet active le service sans le démarrer | `systemctl start kubelet` |
| `TP2.zip` introuvable sur le PC | Fichier enregistré sous `TP2.zip.txt` | Recherche du nom exact, puis `scp` en le renommant |
| Commande du cas 2 sans test de variable | Programme qui sort en erreur dans tous les cas | Ajout du test sur `DB_HOST` en plus de la variable |
| Pas de shell dans le pod etcd | Image etcd 3.6 minimaliste | Appel direct à `etcdctl` / `etcdutl` |
| Fusion des kubeconfig impossible | VM K3s supprimée | Procédure documentée, étape non réalisée |

## Conclusion

Ce TP m'a fait mesurer tout ce que K3s cachait derrière sa commande unique. Monter le cluster pièce par pièce m'a obligé à comprendre pourquoi chaque réglage existe : le swap désactivé pour garder une mémoire prévisible, le même pilote de cgroups côté kubelet et côté containerd, un réseau de pods qui ne chevauche aucun réseau réel, des composants du control plane qui démarrent comme pods statiques avant même que l'API n'existe.

Le cluster obtenu est sain : deux nœuds Ready en v1.34.12, Calico fonctionnel avec un routage natif entre les nœuds (vérifié par une capture réseau), CoreDNS opérationnel. Il est prêt à accueillir l'application fil rouge.

La clinique des pods m'a surtout appris à lire les bons indices au bon endroit. Un problème d'image se voit dans les Events, un crash dans les logs de l'instance précédente et le code de sortie, une configuration manquante dans les Events alors qu'il n'y a aucun log. Elle m'a aussi montré que tous les champs d'un pod ne se modifient pas de la même façon. Enfin, l'exercice du drain a rendu très concret l'intérêt des contrôleurs : sans Deployment, un pod ne survit pas à une simple opération de maintenance.